

Manual del Estudiante: Introducción a la Inteligencia Artificial

Módulo 1: El Mapa de la Inteligencia Artificial

Febrero Tech 2025 — Jaén Tech

Por : Raúl Gómez Téllez

12 de febrero de 2026

Índice general

1. Sesión 1 : Introducción al Machine learning	4
1.1. La Jerarquía: Desambiguación	4
1.1.1. Los tres niveles	4
1.2. Los 4 Paradigmas de Aprendizaje	5
1.2.1. Aprendizaje Supervisado (Supervised Learning)	5
1.2.2. Aprendizaje No Supervisado (Unsupervised Learning)	5
1.2.3. Aprendizaje Semi-Supervisado (Semi-Supervised Learning)	6
1.2.4. Aprendizaje por Refuerzo (Reinforcement Learning)	7
1.3. Taxonomía de Tareas: El "Qué" matemático	7
1.3.1. Predicción (Aprendizaje Supervisado)	7
1.3.2. Descubrimiento de Patrones (Aprendizaje No Supervisado)	8
1.3.3. Generación (IA Generativa)	8
1.4. Áreas de Aplicación: ¿Para qué sirve esto?	9
1.4.1. Visión por Computador (Computer Vision - CV)	9
1.4.2. Procesamiento del Lenguaje Natural (NLP)	9
1.4.3. Predicción Tabular (Industrial AI / Data Science Clásico)	9
1.4.4. IA Multimodal	10
1.5. El Diccionario del Data Scientist	10
1.6. La Ley Suprema: GIGO (Garbage In, Garbage Out)	10
1.7. Tipos de Datos: La Materia Prima	11
1.8. Análisis Exploratorio de Datos (EDA)	11
1.8.1. El Mapa de Calor y las Correlaciones	11
1.8.2. Herramientas de Visualización	12
1.9. Estadística Descriptiva y Escalamiento	13
1.9.1. Varianza y Desviación Típica: La Dispersión	14
1.9.2. El Problema de las Escalas (Feature Scaling)	14
1.9.3. La Solución: Estandarización (Standard Scaler)	14
1.10. Gestión de Datos Desbalanceados (Imbalanced Data)	14
1.10.1. Estrategias de Corrección (Resampling)	15
1.11. La Maldición de la Dimensionalidad y Reducción	15
1.11.1. PCA (Principal Component Analysis)	15
1.11.2. LDA (Linear Discriminant Analysis)	16
1.12. Anatomía de un Algoritmo de ML: Los 3 Pilares	16
1.12.1. Representación (El "Cuerpo")	17
1.12.2. Evaluación (El "Juez")	17
1.12.3. Optimización (El "Motor")	17
1.13. Fundamento Matemático: Regresión Lineal	18
1.13.1. La Ecuación del Hiperplano	18
1.13.2. El Juez: Función de Coste y Convexidad	19
1.14. El Juez: La Función de Coste (Loss Function)	19
1.14.1. Para Regresión (Predecir Cantidades)	19

1.14.2. Para Clasificación (Predecir Probabilidades)	20
1.14.3. Resumen de Elección	21
1.15. Estrategias de Optimización (Los Optimizadores)	21
1.15.1. Variantes Clásicas del Descenso de Gradiente	21
1.15.2. La Evolución Moderna: Adam	22
1.16. Regularización: Controlando la Complejidad	23
1.17. Metodología de Evaluación	23
1.17.1. El Equilibrio: Bias vs Variance	23
1.17.2. El Protocolo de División de Datos	24
1.17.3. Otras técnicas de validación (Cross-Validation)	25
1.18. Métricas de evaluación	26
1.18.1. Desglose de la Matriz (Binaria)	26
1.18.2. Precision vs Recall (El Dilema)	26
1.19. Introducción: Pensamiento Probabilístico	27
1.20. Desglosando el Teorema de Bayes	27
1.21. La Asunción "Naive": Independencia Condicional	28
1.22. Variantes del Algoritmo (Tipos de Datos)	28
1.22.1. Multinomial Naive Bayes (Texto)	28
1.22.2. Bernoulli Naive Bayes (Binario)	28
1.22.3. Gaussian Naive Bayes (Numérico Continuo)	28
1.23. Retos de Ingeniería: El Truco del Logaritmo	29
1.24. El Problema de la Probabilidad Cero (Laplace Smoothing)	29
1.25. Introducción: El Arquitecto Geométrico	29
1.26. El Concepto de Margen Máximo	29
1.26.1. Componentes Clave:	30
1.27. Realidad vs Teoría: Hard Margin vs Soft Margin	30
1.27.1. El Hiperparámetro C (Regularización)	30
1.28. The Kernel Trick: La Magia Dimensional	30
1.28.1. Tipos de Kernels	31
1.29. El Hiperparámetro Gamma (γ)	31
1.30. Resumen de Ajuste (Tuning Guide)	32
2. Sesión 2 : Árboles de decisión,Ensembles y Clustering	33
2.1. Árboles de Decisión (Decision Trees)	33
2.1.1. Fundamentos y Anatomía	33
2.1.2. El Algoritmo CART (Classification and Regression Trees)	33
2.1.3. La familia Quinlan (ID3 y C4.5)	34
2.1.4. Criterios de Impureza (Clasificación)	34
2.1.5. Árboles de Regresión	35
2.1.6. Regularización: El Problema del Overfitting	35
2.1.7. Interpretabilidad: Feature Importance	35
2.2. Métodos de Ensamblaje (Ensemble Learning)	36
2.2.1. Bagging (Bootstrap Aggregating)	36
2.2.2. Boosting (Hypothesis Boosting)	37
2.2.3. Implementaciones Industriales (SOTA)	37
2.2.4. Stacking (Stacked Generalization)	38
2.2.5. Nota de Ingeniería: Estrategias Multiclase (OvR vs OvO)	38
2.3. Aprendizaje No Supervisado (Clustering)	39
2.3.1. K-Means Clustering	39
2.3.2. Clustering Jerárquico (Aglomerativo)	40
2.3.3. DBSCAN (Density-Based Spatial Clustering)	40
2.3.4. Validación de Clústers	41

2.4.	Optimización de Hiperparámetros	42
2.4.1.	Grid Search (Búsqueda en Rejilla)	42
2.4.2.	Random Search (Búsqueda Aleatoria)	42
2.4.3.	Optimización Bayesiana (Optuna / Hyperopt)	42
2.5.	Ingeniería de Producción (Pipelines)	42
2.5.1.	El Concepto de Pipeline	43
2.5.2.	Column Transformer	43
2.5.3.	Serialización (Persistencia del Modelo)	43
2.6.	Procesamiento de Datos Desbalanceados	44
2.6.1.	Técnicas de Re-Muestreo (Resampling)	44
2.6.2.	Generación de Datos Sintéticos (SMOTE)	44
2.6.3.	Class Weights (Pesos de Clase)	44

Capítulo 1

Sesión 1 : Introducción al Machine learning

1.1. La Jerarquía: Desambiguación

Antes de aprender a programar, debemos aprender a hablar con propiedad. En la prensa y el marketing, términos como IA, Machine Learning y Deep Learning se usan indistintamente, pero para un ingeniero representan niveles de abstracción muy diferentes.

Podemos visualizar esto como un sistema de subconjuntos anidados, similar a una **Muñeca Matrioshka** o un Diagrama de Venn concéntrico.

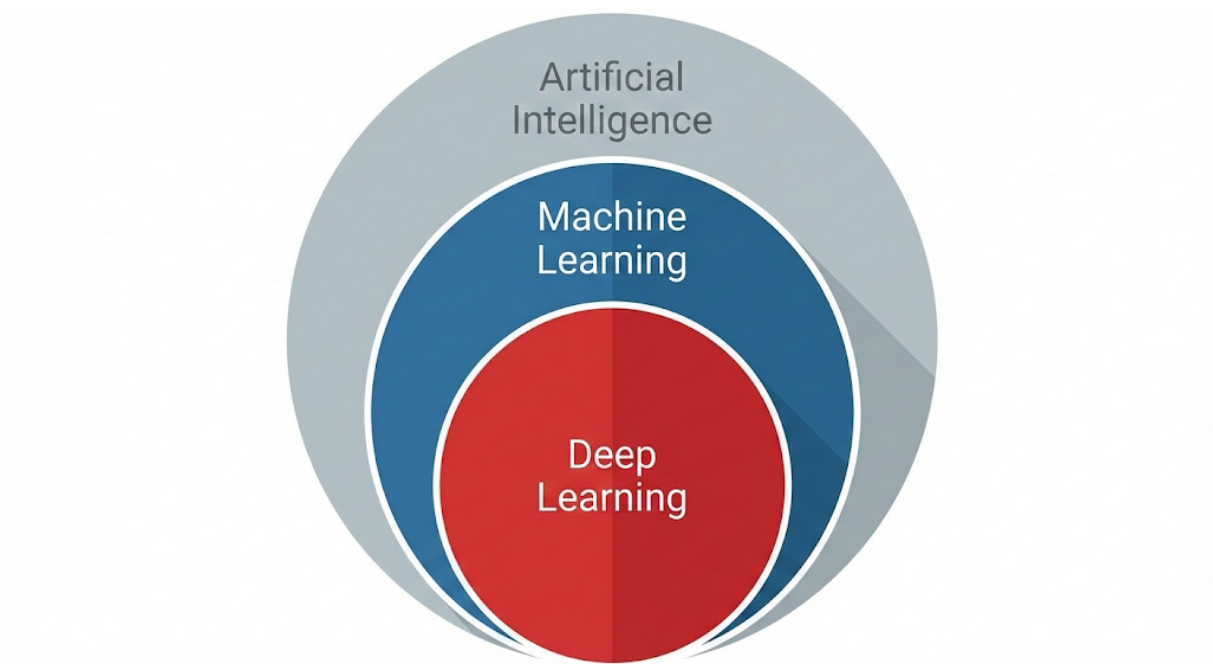


Figura 1.1: Diagrama de Venn del ámbito de la inteligencia artificial

1.1.1. Los tres niveles

- **Inteligencia Artificial (El Círculo Grande):** Es el concepto general que nace en los años 50. Se refiere a cualquier técnica que permita a una máquina imitar el comportamiento inteligente.

Nota: Un programa de ajedrez de 1980 lleno de **if-else** complejos ES Inteli-

gencia Artificial, pero NO es Machine Learning. Es "IA Simbólica" o basada en reglas fijas.

- **Machine Learning (El Círculo Mediano):** Aquí ocurre el cambio de paradigma. En lugar de programar las reglas manualmente, programamos algoritmos que pueden **aprender de los datos**. La máquina deduce las reglas basándose en la experiencia (estadística y probabilidad).
- **Deep Learning (El Círculo Pequeño):** Es una técnica específica dentro del ML inspirada en la estructura del cerebro humano. Utiliza **Redes Neuronales Profundas** (múltiples capas de neuronas artificiales). Es la tecnología necesaria para entender datos complejos como imágenes, audio o texto libre.

1.2. Los 4 Paradigmas de Aprendizaje

Una vez estamos dentro del Machine Learning, ¿cómo aprende exactamente la máquina? Todo depende de qué tipo de datos tengamos y qué feedback reciba el modelo.

1.2.1. Aprendizaje Supervisado (Supervised Learning)

Es la base de la gran mayoría de aplicaciones industriales actuales.

- **Datos:** Tenemos la Entrada (X) y la Respuesta Correcta o Etiqueta (y).
- **Meta:** Encontrar una función que mapee $X \rightarrow y$ para predecir y en datos futuros.
- **Ejemplos:** Filtros de Spam (Email $\rightarrow$ Spam/No Spam), Predicción de precios de casas (Características $\rightarrow$ Precio), Diagnóstico médico (Radiografía $\rightarrow$ Enfermo/Sano).

Analogía: Las Flashcards

Es como estudiar para un examen con tarjetas de memoria. En un lado tienes la pregunta (Input) y en el reverso la respuesta (Label). Lees la pregunta, intentas adivinar, y verificas con el reverso. Si fallas, ajustas tu conocimiento.

1.2.2. Aprendizaje No Supervisado (Unsupervised Learning)

Se utiliza para descubrir lo desconocido.

- **Datos:** Solo tenemos Entradas (X). No hay etiquetas ni "respuestas correctas".
- **Meta:** Encontrar estructuras ocultas, agrupar datos similares o simplificar la información.
- **Ejemplos:** Segmentación de clientes (agrupar compradores por comportamiento), detección de anomalías bancarias.

Analogía: El niño y los LEGOs

Imagina un niño volcando una caja de LEGOs mezclados. Nadie le dice cómo se llaman las piezas, pero él instintivamente empieza a agruparlas: las rojas con las rojas, las pequeñas con las pequeñas. Está haciendo *Clustering* (agrupamiento).

Aprendizaje Auto-Supervisado (Self-Supervised Learning)

Vital para entender la IA Generativa (GPT, Gemini, Llama). Resuelve el problema de "no tener suficientes humanos para etiquetar datos".

- **Datos:** Usamos una parte de la entrada (X) para predecir otra parte de la misma entrada. El dato es su propia etiqueta.
- **Meta:** Aprender el contexto y la estructura profunda del mundo (o del lenguaje).
- **Ejemplos:** Modelos de Lenguaje (LLMs), Pre-entrenamiento de visión artificial.

Analogía: Rellenar los huecos

Si te doy la frase: "El perro persigue al ___ y el gato huye", tu cerebro predice "ratón" o "gato" basándose en el contexto. El sistema oculta partes de la información y trata de recuperarlas. Así aprende a entender el lenguaje leyendo todo Internet sin supervisión humana.

1.2.3. Aprendizaje Semi-Supervisado (Semi-Supervised Learning)

En la industria, rara vez nos encontramos en los extremos perfectos: casi nunca tenemos "todo etiquetado" (demasiado caro) ni "nada etiquetado" (demasiado vago). El escenario más común es tener un pequeño conjunto de datos de alta calidad y una montaña de datos crudos. Aquí brilla el Aprendizaje Semi-Supervisado.

- **El Problema de Negocio:** Etiquetar datos es costoso y lento (requiere humanos expertos), pero recolectar datos crudos es barato y automático.
- **Datos:** Tenemos un conjunto pequeño etiquetado $D_L = \{(x_i, y_i)\}_{i=1}^L$ y un conjunto masivo no etiquetado $D_U = \{x_j\}_{j=1}^U$, donde típicamente $U \gg L$.
- **Meta:** Utilizar la estructura y distribución de D_U para mejorar la generalización del modelo entrenado con D_L .

Analogía: El Profesor y los Exámenes de Práctica

Imagina un profesor que solo corrige 5 exámenes de una clase de 100 alumnos.

- **Supervisado:** El profesor solo usa esos 5 para evaluar el nivel de la clase. (Alta probabilidad de sesgo).
- **Semi-Supervisado:** El profesor corrige los 5 exámenes. Luego, mira las respuestas de los otros 95 alumnos (sin corregirlos). Nota que hay un grupo de 30 alumnos que han respondido idéntico al "Alumno A" (que sacó un 10). Por **continuidad**, el profesor asume que esos 30 alumnos también merecen buena nota y usa esa información para ajustar sus criterios de evaluación global.

Suposiciones Fundamentales (The Cluster Assumption)

Para que esto funcione matemáticamente, el algoritmo asume que los puntos que están cerca en el espacio de características (X) deben compartir la misma etiqueta (y).

- **Separación de Baja Densidad:** El algoritmo intentará colocar la frontera de decisión en "zonas vacías" donde hay pocos datos no etiquetados, evitando cortar a través de un clúster denso de puntos.

Técnicas Comunes en Ingeniería

1. Pseudo-Labeling (Auto-Entrenamiento) Es una técnica iterativa simple pero potente:

1. Entrenamos un modelo inicial solo con los datos etiquetados (D_L).
2. Usamos ese modelo para predecir sobre los datos no etiquetados (D_U).
3. Seleccionamos las predicciones con **alta confianza** (ej. probabilidad $> 0,95$) y las tratamos como si fueran etiquetas reales ("Pseudo-etiquetas").
4. Re-entrenamos el modelo con el dataset combinado ($D_L + D_{Pseudo}$).

Riesgo: Si el modelo inicial es malo, reforzará sus propios errores (Confirmation Bias).

2. Label Propagation (Basado en Grafos) Imagina los datos como nodos en una red. Conectamos los puntos cercanos entre sí.

- "Inyectamos tinta" (la etiqueta) en los nodos etiquetados.
- Dejamos que la tinta se propague a través de las conexiones a los nodos vecinos no etiquetados.
- La etiqueta final de un nodo depende de qué color de tinta" le haya llegado con más fuerza.

1.2.4. Aprendizaje por Refuerzo (Reinforcement Learning)

Basado en la psicología conductista y la toma de decisiones secuenciales.

- **Datos:** Un **Agente** interactúa con un **Entorno**.
- **Ciclo:** Acción $\rightarrow$ Estado $\rightarrow$ Recompensa/Castigo.
- **Meta:** Maximizar la recompensa acumulada a largo plazo.
- **Ejemplos:** Robótica (Boston Dynamics), AlphaGo, conducción autónoma (maniobras complejas).

Analogía: Adiestrar a una mascota

Si el perro se sienta cuando das la orden, recibe una galleta (Refuerzo positivo). Si muerde el sofá, recibe una riña (Refuerzo negativo). El perro aprende la política de acciones que maximiza su número de galletas.

1.3. Taxonomía de Tareas: El "Qué" matemático

Antes de escribir código, el Data Scientist debe hacerse una pregunta fundamental: **¿Qué forma tiene la salida (y) que espero?** Independientemente de si trabajamos con fotos, audios o Excel, casi todos los problemas se reducen a tres operaciones matemáticas fundamentales.

1.3.1. Predicción (Aprendizaje Supervisado)

El objetivo es encontrar una función f tal que $f(X) \approx y$. Dependiendo del tipo de dato de y

Regresión (Salida Continua)

Buscamos predecir una cantidad numérica en una escala infinita o continua.

- **Output Matemático:** Un escalar $y \in \mathbb{R}$ (o un vector de escalares).
- **La Clave:** La distancia entre errores importa. Predecir 100 cuando era 105 es "mejor" que predecir 200.
- **Ejemplos:** Predecir el precio exacto de una acción, la temperatura de un reactor, el tiempo de vida útil restante de una maquinaria (Predictive Maintenance).

Clasificación (Salida Discreta)

Buscamos asignar una etiqueta de un conjunto finito de categorías posibles.

- **Output Matemático:** Un vector de probabilidades que suman 1. (Ej: [0.85 Gato, 0.10 Perro, 0.05 Pájaro]).
- **Tipos:**
 - *Binaria:* Dos clases mutuamente excluyentes (0 o 1). Ej: *Fraude / No Fraude*.
 - *Multiclase:* Más de dos clases, pero solo una correcta. Ej: *Dígitos del 0 al 9*.
 - *Multi-label:* Un dato puede tener varias etiquetas a la vez. Ej: *Una película puede ser "Acción" Y "Comedia"*.

1.3.2. Descubrimiento de Patrones (Aprendizaje No Supervisado)

No existe una "respuesta correcta" (y). Buscamos simplificar o entender la estructura subyacente de X .

Clustering (Agrupamiento)

El algoritmo agrupa los datos basándose en distancias geométricas.

- **Hipótesis:** Los puntos cercanos en el espacio vectorial comparten características similares.
- **Ejemplo:** Segmentación de mercado. El algoritmo descubre que hay un grupo de clientes que "compra mucho los viernes y es joven", sin que nadie le haya definido ese perfil antes.

Reducción de Dimensionalidad

Proyectar datos complejos (muchas columnas) a un espacio más simple (pocas columnas) perdiendo la mínima información posible.

- **Utilidad:** Visualización (pasar de 100 dimensiones a 2D para graficar) y compresión de datos para acelerar otros modelos.

1.3.3. Generación (IA Generativa)

La frontera moderna. Aquí no clasificamos datos existentes, sino que aprendemos la **Distribución de Probabilidad** de los datos de entrenamiento para **muestrear** nuevas instancias. *Ejemplo:* "Imaginar" una cara humana que nunca ha existido, combinando rasgos aprendidos de millones de fotos reales.

1.4. Áreas de Aplicación: ¿Para qué sirve esto?

Una vez definida la tarea matemática, la aplicamos a un dominio específico. Cada dominio requiere una forma distinta de "alimentar" al modelo (Preprocesamiento).

1.4.1. Visión por Computador (Computer Vision - CV)

"Enseñar a la máquina a ver".

- **El Dato (X):** Para la máquina, una imagen no es "arte", es un **Tensor tridimensional** (Alto $\times$ Ancho $\times$ Canales RGB). Una foto de 100x100 píxeles son 30,000 números que entran al modelo.
- **Tareas:**
 - *Image Classification:* ¿Qué hay en la foto?
 - *Object Detection:* ¿Dónde está y qué es? (Dibuja un recuadro/bounding box).
 - *Semantic Segmentation:* Clasifica cada píxel individualmente (vital para coches autónomos para distinguir asfalto de acera).
- **Modelos Rey:** CNNs (Convolutional Neural Networks) y Vision Transformers (ViT).

1.4.2. Procesamiento del Lenguaje Natural (NLP)

"Enseñar a la máquina a leer y escribir".

- **El Dato (X):** El texto es secuencial y ambiguo. Convertimos palabras en vectores numéricos llamados **Embeddings**. En este espacio vectorial, la palabra "Rey" está matemáticamente cerca de "Reina" y "Hombre".
- **Tareas:** Traducción, Resumen automático, Chatbots, Análisis de Sentimiento (Detectar odio en redes sociales).
- **Modelos Rey:** Transformers (Arquitectura base de BERT, GPT, Llama). Su clave es el mecanismo de "Atención": saber qué palabras anteriores son relevantes para entender la actual.

1.4.3. Predicción Tabular (Industrial AI / Data Science Clásico)

"La IA de las empresas".

- **El Dato (X):** Datos estructurados en filas y columnas (SQL, Excel). Mezclan números (Edad) con categorías (Ciudad) y fechas. Suelen estar sucios (valores nulos).
- **El Reto:** A diferencia de imágenes o texto, aquí las columnas no tienen relación espacial entre sí. (La columna 1 no tiene por qué estar relacionada con la 2).
- **Modelos Rey:** Aquí el Deep Learning NO suele ser el ganador. Dominan los modelos de ensamblaje de árboles: **XGBoost, LightGBM, CatBoost**.
- **Uso:** Detección de fraude, Scoring de crédito, Previsión de demanda (Retail), Churn Prediction (¿Qué cliente se va a dar de baja?).

1.4.4. IA Multimodal

La convergencia final. Modelos capaces de proyectar imágenes y texto al **mismo espacio vectorial**. Esto nos permite hacer cosas como: "Buscar en Google Fotos poniendo 'perro en la playa'" (El modelo entiende que el texto 'perro' y los píxeles de un perro son el mismo concepto matemático).

1.5. El Diccionario del Data Scientist

Vamos a formalizar los términos que utilizaremos en `scikit-learn`:

- **Dataset:** Nuestra colección total de información bruta.
- **Parámetros (Internos):** Son las variables que el modelo **aprende automáticamente** de los datos durante el entrenamiento (ej. los pesos w o la pendiente de la recta).
- **Hiperparámetros (Externos):** Son los valores ajustables que **tú defines antes de entrenar** para controlar el comportamiento y la complejidad del algoritmo. No se aprenden, se configuran. *Ejemplos: Número de neuronas, Learning Rate, cantidad de vecinos (k), profundidad del árbol.*
- **Instancia (Sample):** Cada una de las filas del dataset. Representa una unidad de observación (un cliente, una flor, una imagen).
- **Features (X) (Características/Atributos):** Son las variables independientes. Las "pistas" que damos al modelo.
 - *Númericas:* Edad, salario, altura. (El modelo las entiende nativamente).
 - *Catóricas:* Color (Rojo/Azul), Ciudad (Jaén/Madrid). (Debemos transformarlas a números mediante *Encoding*).
 - **Notación:** Se usa X mayúscula porque matemáticamente es una **Matriz** (Filas $\times$ Columnas).
- **Target (y) (Variable Objetivo/Etiqueta):** Es la variable dependiente. Lo que queremos predecir.
 - **Notación:** Se usa y minúscula porque es un **Vector** (una sola columna).
- **Bias (Sesgo del Modelo):** Es el error introducido por aproximar un problema del mundo real (que puede ser muy complejo) con un modelo demasiado simple (ej. usar una línea recta para datos curvos). *Consecuencia:* Alto Bias $\rightarrow$ **Underfitting**.
- **Variance (Varianza del Modelo):** Es la sensibilidad del modelo a pequeñas fluctuaciones en el conjunto de entrenamiento. Si cambias ligeramente los datos de entrenamiento y el modelo cambia drásticamente, tiene alta varianza. Significa que está memorizando el ruido. *Consecuencia:* Alta Varianza $\rightarrow$ **Overfitting**.

1.6. La Ley Suprema: GIGO (Garbage In, Garbage Out)

Antes de hablar de algoritmos, debemos grabar a fuego la regla de oro del Data Science: **"Basura entra, Basura sale" (Garbage In Garbage Out)**.

Muchos creen que un modelo de Deep Learning de última generación puede hacer magia con cualquier tipo de dato. Es falso. La jerarquía de valor en un proyecto de IA es la siguiente:

Datos Excelentes + Modelo Simple $\gg$ Datos Mediocres + Modelo Complejo

Si alimentas al modelo con datos ruidosos, mal etiquetados o sesgados, el modelo aprenderá ruido y sesgo con una confianza matemática absoluta.

El Peligro del Sesgo (Bias)

Los datos históricos suelen contener los prejuicios de la sociedad. Si entrenas una IA para seleccionar currículums basándote en contrataciones de los últimos 20 años (donde mayoritariamente se contrataban hombres), el modelo aprenderá matemáticamente que "Ser mujer penaliza la puntuación". El modelo no es machista; los datos lo son, y el algoritmo simplemente es un espejo.

1.7. Tipos de Datos: La Materia Prima

El algoritmo que elijas dependerá estrictamente de la forma de tus datos.

- **Datos Tabulares (Estructurados):** Filas y columnas (Excel, SQL, CSV). Cada columna es una característica clara (Edad, Precio, Categoría). Es el dominio del *Machine Learning Clásico*.
- **Datos No Estructurados:** No tienen un formato de tabla predefinido. Imágenes (matriz de píxeles), Texto libre, Audio. Para que una máquina entienda esto, generalmente necesitamos *Deep Learning*.

1.8. Análisis Exploratorio de Datos (EDA)

Antes de entrenar nada, debemos ponernos el sombrero de detective. El EDA no es un paso opcional; es donde descubres si tu proyecto es viable. El objetivo es responder: *¿Cómo son mis datos? ¿Hay relaciones ocultas? ¿Hay errores graves?*

1.8.1. El Mapa de Calor y las Correlaciones

La correlación mide la fuerza de la relación entre dos variables. Va de -1 a +1.

- **+1 (Correlación Positiva Perfecta):** Si A sube, B sube. (Ej. Estatura y Peso).
- **0 (Sin Correlación):** El caos. (Ej. Tu número de zapato y la nota del examen).
- **-1 (Correlación Negativa Perfecta):** Si A sube, B baja. (Ej. Velocidad del coche y tiempo en llegar).

Tipos de Correlación

- **Pearson (r):** La estándar. Mide relaciones **Lineales**.
- **Spearman (ρ):** Mide relaciones **Monótonas** (de rango). Es útil si la relación es curva o si hay outliers, ya que no le importan los valores exactos, solo el orden.

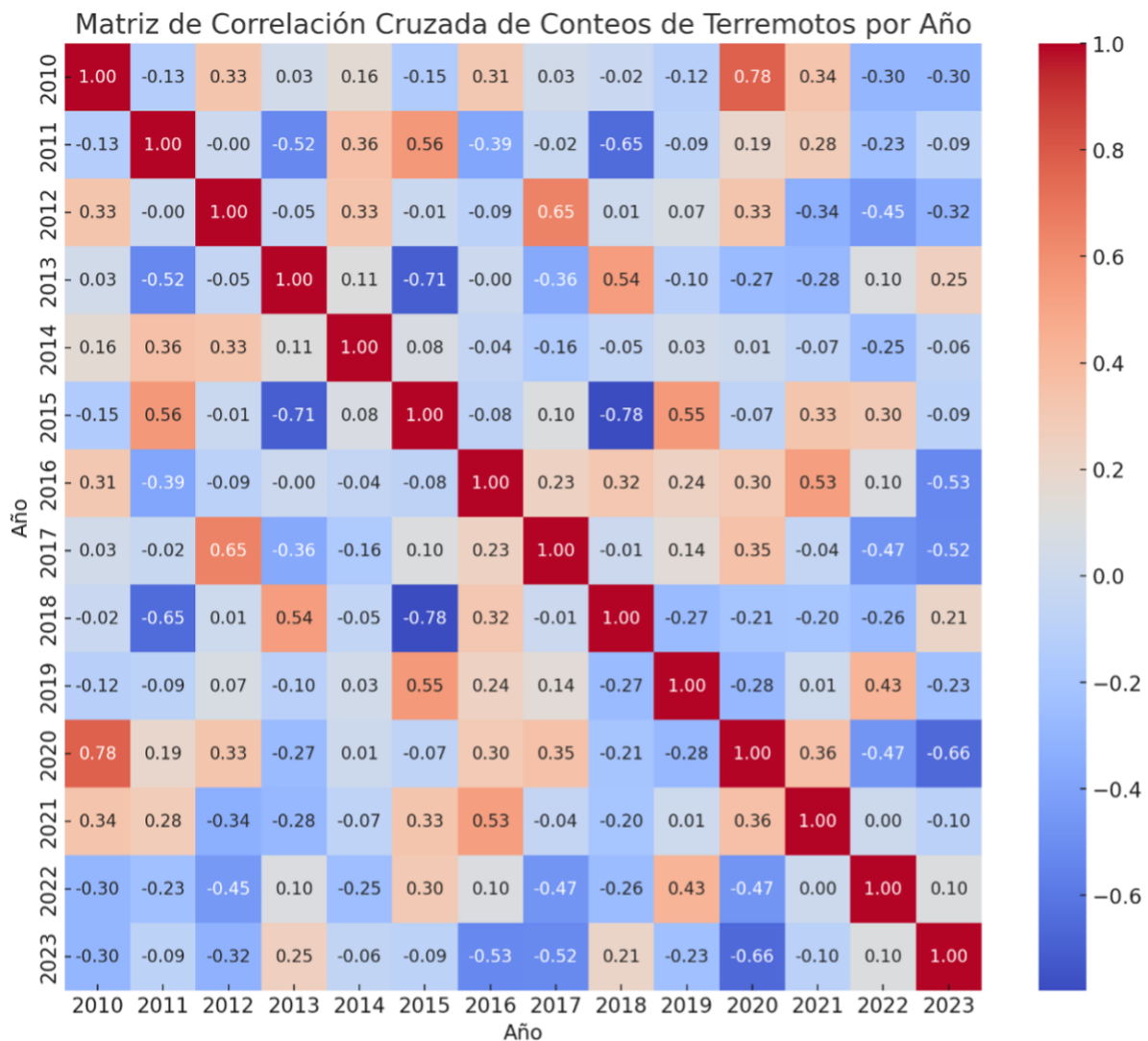


Figura 1.2: Ejemplo de matriz de correlación

La Trampa de la Multicolinealidad

Podrías pensar que tener muchas variables correlacionadas con el objetivo es bueno. Pero, ¿qué pasa si las variables (X) están correlacionadas **entre sí**? Ejemplo: Predecir el precio de una casa usando "Superficie en m^2 " "Superficie en $pies^2$ ". **Problema:** Ambas columnas dicen lo mismo. Esto confunde al modelo (especialmente Regresión Lineal), vuelve inestables los pesos y aumenta el coste computacional. **Solución:** Si dos features tienen una correlación $> 0,95$, **borra una de las dos**.

1.8.2. Herramientas de Visualización

No te fíes solo de los números. El *Cuarteto de Anscombe* demuestra que cuatro datasets pueden tener la misma media y varianza, pero formas totalmente distintas. ¡Hay que pintar los datos!

Histograma y KDE (Análisis Univariante)

¿Qué forma tiene una sola variable? Nos permite ver la **Distribución**.

- ¿Es una Campana de Gauss (Normal)? → Genial para la mayoría de modelos.
- ¿Está sesgada a la izquierda/derecha? (Skewness) → Quizás necesitemos aplicar una transformación logarítmica ($\log(x)$) para enderezarla.

Scatter Plot (Análisis Bivariante)

¿Cómo interactúan dos variables? Pintamos X_1 vs X_2 (o X vs y).

- Buscamos formas: ¿Forman una línea? ¿Una curva? ¿Nubes separadas (clusters)?
- Es la mejor forma visual de detectar si un problema es Lineal o No Lineal.

Box Plot (Diagrama de Caja y Bigotes)

El detector de mentiras. Resume la distribución en 5 números: Mínimo, Cuartil 1 (25 %), Mediana, Cuartil 3 (75 %) y Máximo.

Profundidad Técnica: Detección de Outliers con IQR

Los Box Plots visualizan los datos atípicos como puntos sueltos fuera de los "bigotes". Matemáticamente, un dato es un outlier si está fuera del rango:

$$[Q1 - 1,5 \cdot IQR, \quad Q3 + 1,5 \cdot IQR]$$

Donde $IQR = Q3 - Q1$. Esta regla es el estándar de oro en la industria para limpiar datos automáticamente.

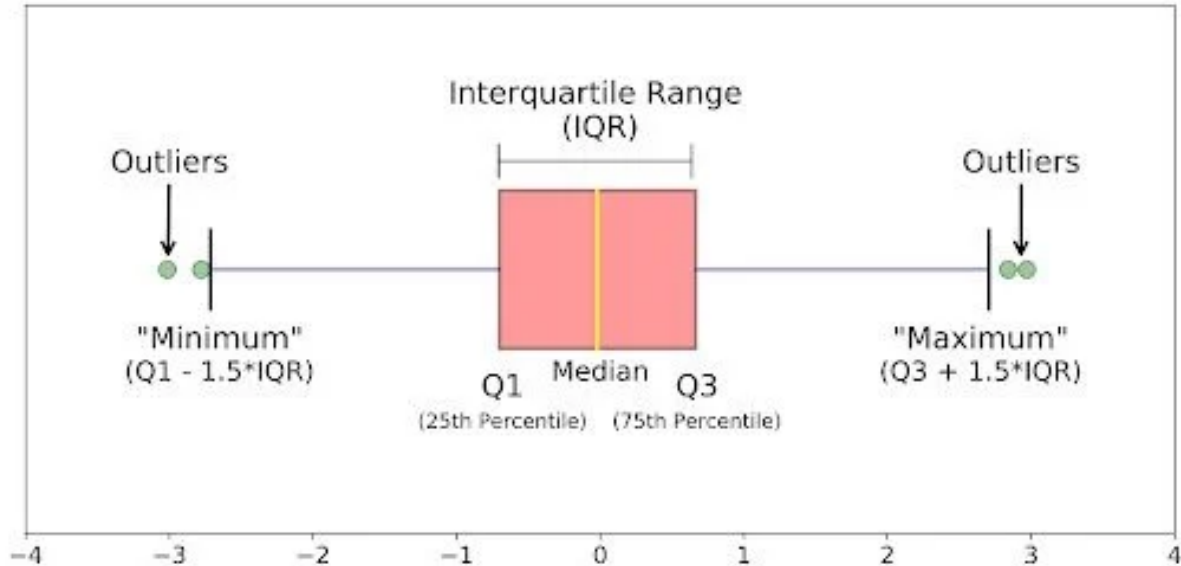


Figura 1.3: Diagrama de caja y bigotes

1.9. Estadística Descriptiva y Escalamiento

Para entender nuestros datos, no basta con mirar las filas; necesitamos métricas que resuman su comportamiento. Aquí entran en juego conceptos que afectan drásticamente al rendimiento de modelos basados en distancias como SVM, K-Nearest Neighbors o el propio Descenso del Gradiente.

1.9.1. Varianza y Desviación Típica: La Dispersión

Imagina que quieres medir la altura de dos grupos de personas.

- **Grupo A:** Todos miden exactamente 1.75m. (Cero dispersión).
- **Grupo B:** Hay bebés, adultos y jugadores de la NBA. La media también es 1.75m, pero los datos son radicalmente distintos.
- **Varianza (σ^2):** Mide qué tan dispersos están los datos respecto a su media. Matemáticamente, es el promedio de las distancias al cuadrado.

$$\sigma^2 = \frac{\sum (x_i - \mu)^2}{N}$$

- **Desviación Típica (σ):** Es la raíz cuadrada de la varianza. *¿Por qué es más útil?* Porque recupera las unidades originales. Si medimos altura en metros, la Varianza está en m^2 (poco intuitivo), mientras que la Desviación Típica está en metros.

1.9.2. El Problema de las Escalas (Feature Scaling)

Este es uno de los errores silenciosos más comunes. Imagina un dataset para predecir precios de casas con dos columnas:

1. **Número de Habitaciones:** Rango de 1 a 5.
2. **Precio en Euros:** Rango de 100,000 a 1,000,000.

Para un algoritmo que calcula distancias euclidianas (como KNN o SVM), una diferencia de 1 habitación es despreciable (1 unidad), mientras que una diferencia de precio es gigante (miles de unidades). El modelo **se volverá ciego** al número de habitaciones porque el precio domina matemáticamente el cálculo.

El Impacto en el Gradiente

Si no escalas tus datos, la "montaña" de la función de coste se deforma, volviéndose un valle muy estrecho y alargado. El algoritmo de *Gradient Descent* tardará muchísimo más en converger porque tendrá que dar pasos en zig-zag muy ineficientes.

1.9.3. La Solución: Estandarización (Standard Scaler)

Debemos poner todas las columnas en la misma "regla de medir". Aunque existe la *Normalización Min-Max* (forzar todo entre 0 y 1), en la industria solemos preferir la **Estandarización (Z-Score)**.

Transformamos los datos para que tengan una **Media (μ) de 0** y una **Desviación Típica (σ) de 1**.

$$z = \frac{x - \mu}{\sigma}$$

De esta forma, tanto las "Habitaciones" como el "Precio" oscilarán mayoritariamente entre -3 y +3 desviaciones estándar.

1.10. Gestión de Datos Desbalanceados (Imbalanced Data)

Uno de los problemas más comunes en la vida real. Imagina un dataset de detección de cáncer donde el 99 % de pacientes están sanos (Clase 0) y el 1 % tienen cáncer (Clase 1). Si el modelo decide ser "perezoso" y predecir siempre "Sano", tendrá un 99 % de Accuracy (Precisión), pero su utilidad será nula.

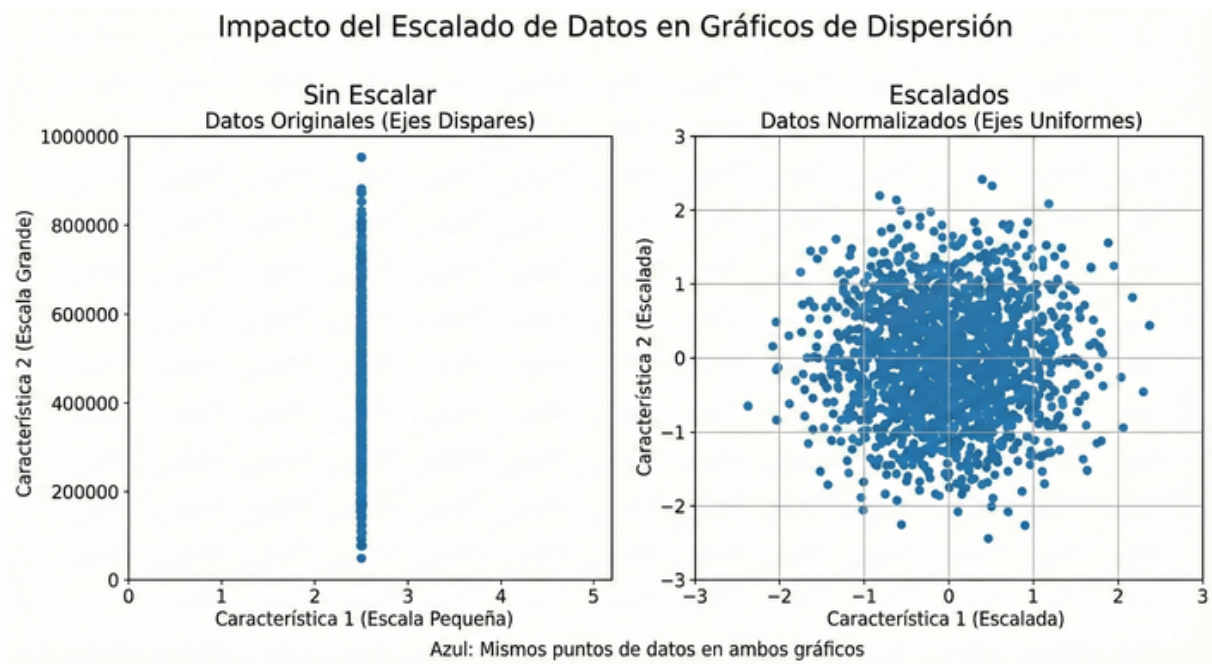


Figura 1.4: Comparativa entre escalar y no escalar

1.10.1. Estrategias de Corrección (Resampling)

- **Undersampling (Submuestreo):** Eliminamos aleatoriamente ejemplos de la clase mayoritaria (Sanos) hasta igualar a la minoritaria. *Problema:* Estamos tirando datos a la basura. Aumenta la varianza del modelo.
- **Oversampling (Sobremuestreo):** Duplicamos aleatoriamente ejemplos de la clase minoritaria (Enfermos). *Problema:* El modelo ve la misma foto exacta muchas veces, lo que lleva al Overfitting extremo.
- **SMOTE (Synthetic Minority Over-sampling Technique):** La solución profesional. En lugar de duplicar, SMOTE mira un punto de la clase minoritaria, busca sus vecinos más cercanos (k-Nearest Neighbors) y **crea matemáticamente un punto nuevo** en la línea que los une. Inventa datos plausibles que no existían antes.

1.11. La Maldición de la Dimensionalidad y Reducción

Añadir más columnas (*Features*) no siempre es mejor. A medida que aumentan las dimensiones, el volumen del espacio crece exponencialmente y los datos se vuelven "dispersos".

- En 1 dimensión (línea), encontrar a tu vecino es fácil.
- En 100 dimensiones, la distancia media entre dos puntos cualquiera se vuelve enorme y casi idéntica. **La distancia euclidiana deja de tener sentido.**

Para combatirlo, usamos técnicas de Reducción de Dimensionalidad (Feature Extraction).

1.11.1. PCA (Principal Component Analysis)

Enfoque: No Supervisado (Ignora las etiquetas y)

PCA busca simplificar la complejidad proyectando los datos en menos dimensiones, preservando la estructura global.

- **El criterio:** Maximizar la **Varianza**. PCA busca el ángulo desde el cual los datos están más "dispersos" (donde hay más información y menos solapamiento).

Analogía: El fotógrafo

Si quieres tomar una foto a un grupo de personas para identificarlas, no tomas la foto desde arriba (verías solo cabezas = poca varianza). La tomas de frente. PCA encuentra ese "frente" matemáticamente.

1.11.2. LDA (Linear Discriminant Analysis)

Enfoque: Supervisado (Utiliza las etiquetas y)

A diferencia de PCA, a LDA no le importa la forma global de la nube de puntos, sino qué hace diferentes a las clases A y B.

- **El criterio:** Maximizar la **Separación entre Clases** (Between-Class Variance) y minimizar la dispersión interna de cada clase (Within-Class Variance).
- **Objetivo:** Busca el eje que logra que los puntos rojos estén lo más lejos posible de los azules, y a la vez que los rojos estén muy juntos entre sí.

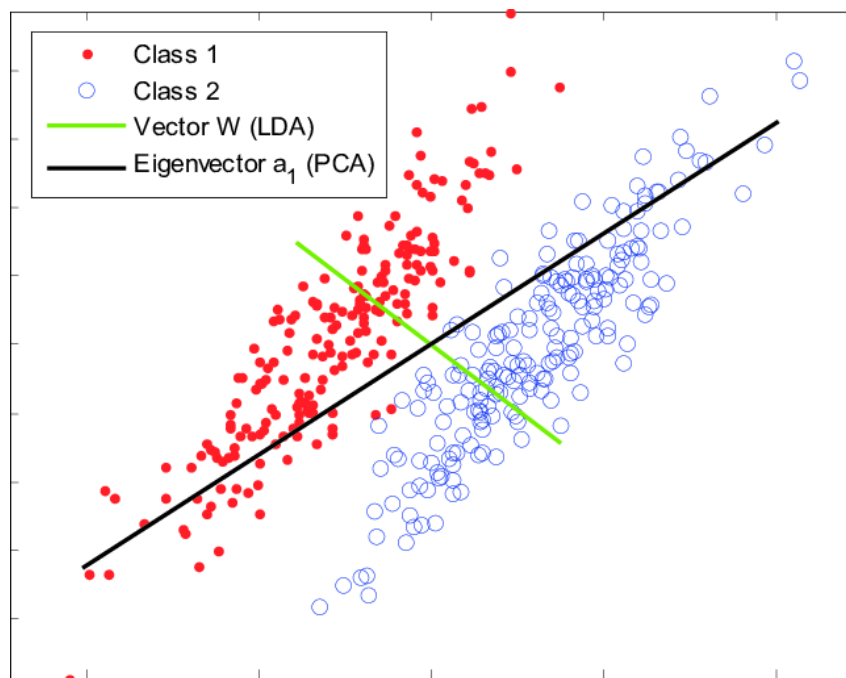


Figura 1.5: Comparativa entre separación por LDA y PCA

1.12. Anatomía de un Algoritmo de ML: Los 3 Pilares

A menudo tratamos a los algoritmos (como una Red Neuronal o un SVM) como cajas negras monolíticas. Sin embargo, desde el punto de vista de la ingeniería, cualquier algoritmo de Machine Learning se puede deconstruir en tres componentes intercambiables.

Si entiendes estas tres piezas, puedes entender (e inventar) cualquier algoritmo.

1.12.1. Representación (El "Cuerpo")

¿Qué aspecto tiene el modelo y qué es capaz de aprender?

Es el conjunto de clasificadores o funciones que el algoritmo puede utilizar. Define el **Espacio de Hipótesis**.

- **Función:** Traducir los datos de entrada a una estructura matemática. Si elegimos una representación errónea, el modelo jamás podrá aprender, por muchos datos que tenga.
- **El Límite:** Si tu representación es una "Línea Recta" (Regresión Lineal), el modelo es físicamente incapaz de entender un patrón circular.
- **Ejemplos:**
 - *Instancias:* K-Nearest Neighbors (El modelo es los propios datos).
 - *Hiperplanos:* SVM, Regresión Logística.
 - *Árboles:* Decision Trees, Random Forest.
 - *Grafos de Capas:* Redes Neuronales.

1.12.2. Evaluación (El "Juez")

¿Cómo distinguimos un modelo bueno de uno malo?

El modelo necesita una métrica interna para saber si va por buen camino durante el entrenamiento. Es la **Función de Objetivo** o Función de Coste.

- **Función:** Convertir la "calidad" de la predicción en un solo número (escalar) que podamos minimizar o maximizar.
- **Importante:** A veces la métrica de evaluación interna (ej. Log Loss) es diferente a la métrica externa que enseñamos al cliente (ej. Accuracy), porque necesitamos que sea matemáticamente suave (diferenciable) para poder optimizarla.
- **Ejemplos:**
 - *Error Cuadrático (MSE):* Para regresión.
 - *Entropía Cruzada (Log Loss):* Para clasificación probabilística.
 - *Hinge Loss:* Para SVM (Margen Máximo).

1.12.3. Optimización (El "Motor")

¿Cómo buscamos la mejor configuración posible?

Una vez tenemos la estructura (Representación) y sabemos medir el error (Evaluación), necesitamos un método para buscar entre las infinitas combinaciones de parámetros aquella que dé el menor error.

- **Función:** Ajustar los pesos (w) iterativamente para mejorar la puntuación de la evaluación.
- **Ejemplos:**
 - *Combinatoria:* Búsqueda Voraz (Greedy Search) usada en Árboles de Decisión.
 - *Continua:* Descenso del Gradiente (SGD, Adam) usado en Redes Neuronales y Regresión.
 - *Restringida:* Programación Cuadrática (usada en SVM).

Analogía: El Piloto de Carreras

Para ganar una carrera necesitas tres cosas:

1. **El Coche (Representación):** ¿Llevas un Ferrari o un Tractor? Esto define tu potencial máximo (si llevas un tractor, nunca ganarás en F1).
2. **El Cronómetro (Evaluación):** Necesitas medir tu tiempo por vuelta. Si no sabes si vas rápido o lento, no puedes mejorar.
3. **El Piloto (Optimización):** Es quien mueve el volante y los pedales basándose en el tiempo del cronómetro para ir cada vez más rápido.

Profundidad Técnica: El Efecto Mix & Match

Lo fascinante del ML es que puedes mezclar piezas.

- Si tomas una **Línea** (Representación) + **Error Cuadrático** (Evaluación) + **Gradiente** (Optimización) = **Regresión Lineal**.
- Si a esa misma **Línea** le cambias la evaluación por **Log Loss** = **Regresión Logística**.
- Si cambias la evaluación por **Hinge Loss** = **SVM Lineal**.

¡Es el mismo "cuerpo" (línea recta) entrenado con "jueces" distintos!

1.13. Fundamento Matemático: Regresión Lineal

El modelo paramétrico más fundamental en aprendizaje automático es la Regresión Lineal. Su premisa es que la variable objetivo y puede aproximarse mediante una combinación lineal de las características de entrada X .

Aunque en educación básica se enseña como $y = mx + b$ (univariante), en problemas reales trabajamos en espacios de alta dimensionalidad ($\mathbb{R}^n$).

1.13.1. La Ecuación del Hiperplano

Matemáticamente, definimos la predicción $\hat{y}$ como el producto punto entre el vector de características y el vector de pesos:

$$\hat{y} = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n = \mathbf{w}^T \mathbf{x} + b$$

- $\mathbf{x} \in \mathbb{R}^n$: Vector de características (Features). Representa la entrada (ej. metros cuadrados, número de habitaciones).
- $\mathbf{w} \in \mathbb{R}^n$: Vector de pesos (Weights). Son los parámetros aprendibles que determinan la influencia de cada característica.
- $b \in \mathbb{R}$: Sesgo (Bias). Es el término independiente (intercepto) que permite desplazar el hiperplano del origen.

Analogía: El Ingeniero de Mezcla (Suma Ponderada)

Podemos visualizar el vector $\mathbf{w}$ como una mesa de mezclas de audio.

- Las pistas de audio de entrada son las características $\mathbf{x}$ (Batería, Voz, Guitarra).
- Los faders o deslizadores son los pesos $\mathbf{w}$.
- **El proceso de entrenamiento** consiste en ajustar los niveles de los faders para minimizar la distorsión.
- *Interpretación:* Si la característica x_3 (Ruido) perjudica la predicción, el modelo aprenderá a bajar su fader ($w_3 \rightarrow 0$). Si x_1 es crítica, aumentará su ganancia.

1.13.2. El Juez: Función de Coste y Convexidad

Para entrenar este sistema, necesitamos cuantificar la discrepancia entre la predicción y la realidad. Utilizamos el **Error Cuadrático Medio (MSE)**:

$$J(\mathbf{w}) = \frac{1}{N} \sum_{i=1}^N (y^{(i)} - \hat{y}^{(i)})^2$$

Justificación del Cuadrado

El uso del término cuadrático no es arbitrario; obedece a dos razones críticas de ingeniería:

1. **Penalización No Lineal:** Un error de 10 unidades tiene un coste 100 veces mayor que un error de 1 unidad. Esto fuerza al modelo a priorizar la corrección de los valores atípicos (outliers) o fallos graves.
2. **Diferenciabilidad:** A diferencia del valor absoluto, la función cuadrática es suave (diferenciable en todo su dominio), lo que facilita el cálculo del gradiente.

Profundidad Técnica: Garantía de Convergencia: Convexidad

Una propiedad única de la Regresión Lineal con MSE es que la superficie de error $J(\mathbf{w})$ es **estrictamente convexa**. Geométricamente, esto significa que tiene forma de "cuenco" perfecto con un **único mínimo global**. **Implicación:** No existen mínimos locales. Cualquier algoritmo de optimización basado en gradiente (como Gradient Descent), si se configura correctamente, tiene la garantía matemática de converger a la solución óptima.

1.14. El Juez: La Función de Coste (Loss Function)

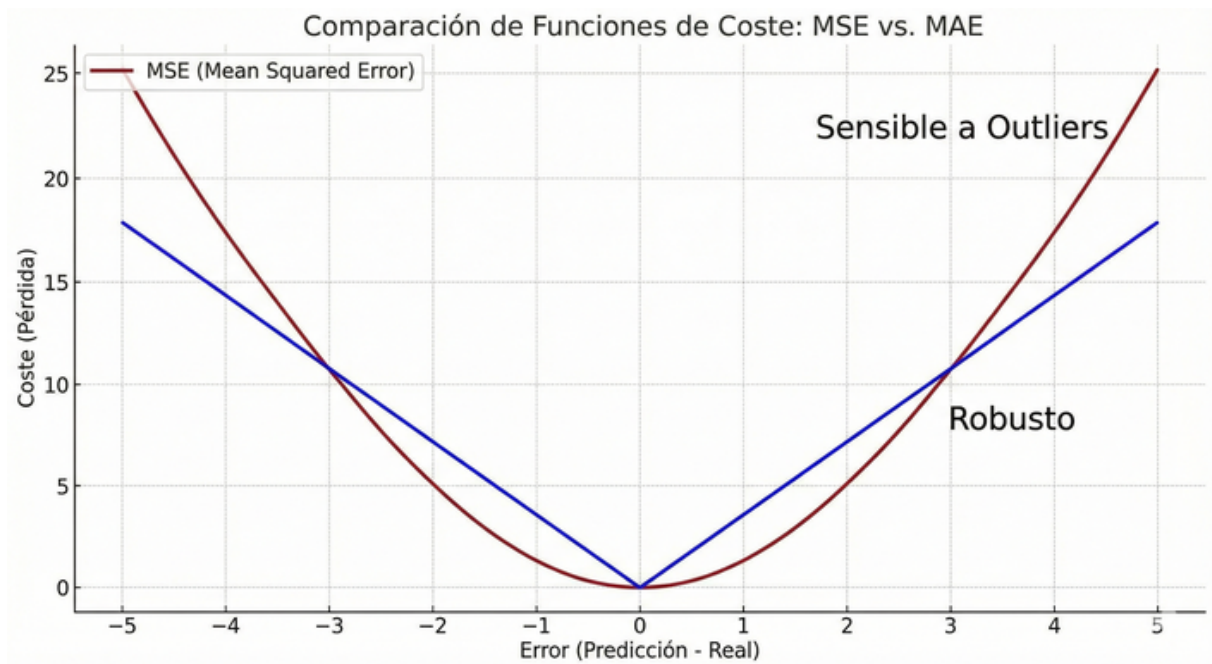
Si el algoritmo es el coche y el optimizador es el motor, la **Función de Coste (J)** es el GPS. Es la fórmula matemática que le dice al modelo: *"Te has equivocado por 100 metros"*.

El objetivo de **todo** entrenamiento en Machine Learning es simple: encontrar los parámetros w que hacen que el valor de esta función sea lo más cercano a 0 posible (Minimización).

Pero no existe un juez universal. Dependiendo de la tarea, usamos reglas distintas.

1.14.1. Para Regresión (Predecir Cantidades)

Cuando la salida es un número continuo (ej. precio de una casa), medimos la distancia física entre la predicción y la realidad.



MSE (Mean Squared Error) - El Estándar

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Calculamos los residuos (errores), los elevamos al cuadrado y hacemos la media.

- **Propiedad Clave (Convexidad):** Al elevar al cuadrado, la gráfica del error forma una parábola perfecta (un cuenco). Esto garantiza que si soltamos una pelota (descenso del gradiente), siempre caerá al fondo único.
- **Castigo a Outliers:** Un error de 10 unidades se convierte en un coste de 100. El modelo **odia** equivocarse por mucho, así que se esforzará desproporcionadamente en corregir los datos atípicos (outliers).

MAE (Mean Absolute Error) - El Robusto

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

Usamos el valor absoluto. El castigo es lineal.

- **Propiedad Clave:** Es más estable ante datos sucios. Si tienes un dato erróneo (un outlier gigante), el MAE no entra en pánico como el MSE.

1.14.2. Para Clasificación (Predecir Probabilidades)

Si estamos clasificando (SÍ/NO), el MSE no sirve. Imagina que el modelo predice que la probabilidad de cáncer es 0.01 (muy seguro de que NO), pero el paciente sí tiene cáncer (1.0). El error matemático es pequeño (0.99), pero el error conceptual es fatal: **el modelo estaba seguro de algo falso**.

Para esto usamos la **Log Loss (Entropía Cruzada)**.

Binary Cross-Entropy

$$J = -\frac{1}{N} \sum [y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$$

Esta fórmula utiliza logaritmos para penalizar la confianza equivocada.

Analogía: El Castigo a la Arrogancia

- Si predices 0.9 y es 1 (Acierto confiado) → Coste casi 0.
- Si predices 0.6 y es 1 (Duda razonable) → Coste bajo.
- Si predices 0.0001 y es 1 (**Arrogancia equivocada**) → **Coste INFINITO**.

Como el $\log(0)$ tiende a infinito, esta función destruye al modelo si este dice "Es imposible" y resulta ser verdad. Le obliga a nunca ser dogmático a menos que tenga pruebas perfectas.

1.14.3. Resumen de Elección

Problema	Función Recomendada	Por qué
Regresión (Estándar)	MSE	Fácil de derivar, solución única.
Regresión (Datos Sucios)	MAE	Ignora mejor los outliers.
Clasificación Binaria	Binary Cross-Entropy	Penaliza probabilidades incorrectas.
Clasificación Múltiple	Categorical Cross-Entropy	Generalización para varias clases.

1.15. Estrategias de Optimización (Los Optimizadores)

Ya sabemos que el objetivo es minimizar la función de coste y que la herramienta matemática es el gradiente. Pero, ¿cómo ejecutamos ese descenso paso a paso?

La regla básica de actualización de pesos es:

$$w_{nuevo} = w_{viejo} - \alpha \cdot \nabla J(w)$$

Donde $\nabla J(w)$ es el gradiente (la pendiente). La diferencia entre los distintos optimizadores radica en **cuántos datos miramos** para calcular ese gradiente y **cómo adaptamos** el paso.

1.15.1. Variantes Clásicas del Descenso de Gradiente

Existen tres estrategias principales basadas en la cantidad de datos utilizados:

Batch Gradient Descent (El Prudente)

Utiliza **TODO** el dataset para calcular el error promedio y luego da **UN solo paso**.

- **Pro:** El cálculo del gradiente es perfecto. La trayectoria hacia el mínimo es una línea recta y suave.
- **Contra:** Es computacionalmente inviable con Big Data. Si tienes 10 millones de datos, tienes que leerlos todos solo para moverte un milímetro.

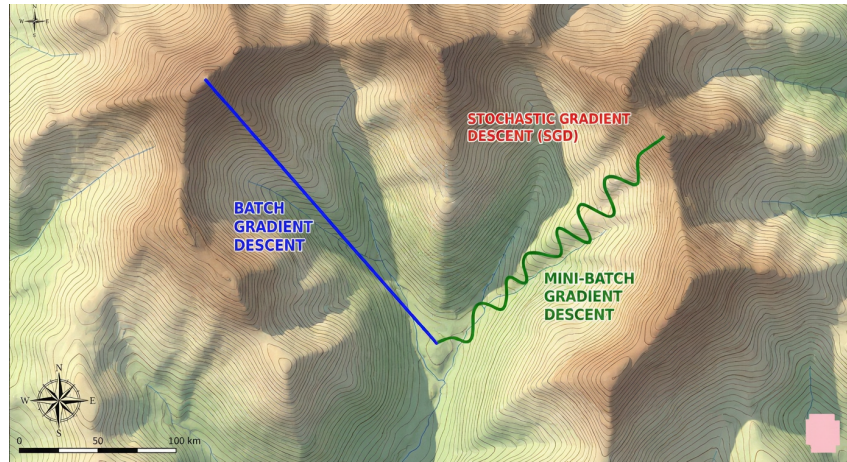


Figura 1.6: Comparativa entre los distintos algoritmos de descenso del gradiente

Stochastic Gradient Descent - SGD (El Borracho)

Toma **UN solo dato** al azar, calcula el error de ese único dato, y da un paso inmediato.

- **Pro:** Extremadamente rápido. Aprende "al vuelo".
- **Contra:** Es muy ruidoso. Como cada dato es diferente (y puede ser un outlier), el camino hacia el mínimo es un zig-zag caótico ("Drunkard's walk"). Nunca converge perfectamente, siempre oscila alrededor del fondo.

Mini-Batch Gradient Descent (El Estándar)

El punto medio y el estándar de la industria. Toma un pequeño lote (ej. 32, 64 o 128 datos), calcula el error promedio de ese lote y da un paso.

- **Pro:** Reduce el ruido del SGD pero mantiene la velocidad. Además, aprovecha la arquitectura paralela de las GPUs (que están diseñadas para multiplicar matrices de ese tamaño).

1.15.2. La Evolución Moderna: Adam

Si el SGD es un borracho y el Mini-Batch es un caminante estándar, **Adam (Adaptive Moment Estimation)** es una bola pesada de metal rodando cuesta abajo con un sistema de frenos inteligente.

Es el optimizador por defecto en Deep Learning porque combina dos "superpoderes" matemáticos que solucionan los problemas del descenso clásico:

1. **Momentum (Inercia):** Imagina que la función de coste es un barranco irregular con pequeños baches (mínimos locales).
 - El SGD normal se quedaría atrapado en cada pequeño bache.
 - Adam acumula "velocidad" (promedio móvil de gradientes pasados). Si viene bajando rápido, la inercia le permite pasar por encima de los pequeños baches sin detenerse, buscando el valle profundo real.
2. **Adaptive Learning Rate (RMSProp):** No todas las características (w) son iguales. Algunas aparecen mucho (frecuentes) y otras casi nunca (raras).
 - Adam ajusta el tamaño del paso (α) **individualmente para cada peso**.

- Si un peso apenas se mueve, Adam le da "pasos de gigante" para acelerarlo. Si otro peso es muy inestable, Adam reduce su paso para frenarlo y estabilizarlo.

Profundidad Técnica: ¿Por qué usar Adam?

En la práctica, Adam suele funcionar bien "out-of-the-box" (sin tocar nada). Mientras que en SGD tienes que probar manualmente muchos valores para el *Learning Rate* (α) para que el modelo no explote, Adam es tolerante, robusto y converge mucho más rápido.

1.16. Regularización: Controlando la Complejidad

A veces, el modelo se "aprende de memoria" el ruido de los datos de entrenamiento (Overfitting). Matemáticamente, esto se ve como pesos (w) extremadamente grandes y específicos.

Para evitarlo, alteramos la Función de Coste añadiendo un término de penalización.

$$Coste_{total} = Error(MSE) + \lambda \cdot (TamW)$$

Donde λ (Lambda) es la fuerza del freno.

- **Regularización L1 (Lasso):** Intenta volver cero los pesos de las características inútiles. Sirve para **selección automática de características**.
- **Regularización L2 (Ridge):** Intenta que todos los pesos sean pequeños pero no cero. Es la más común para evitar que una sola variable domine la predicción.

Intuición

Le decimos al modelo: "Tu trabajo es minimizar el error, PERO te voy a multar por cada peso alto que uses". El modelo se ve obligado a buscar la solución más simple posible que explique bien los datos.

1.17. Metodología de Evaluación

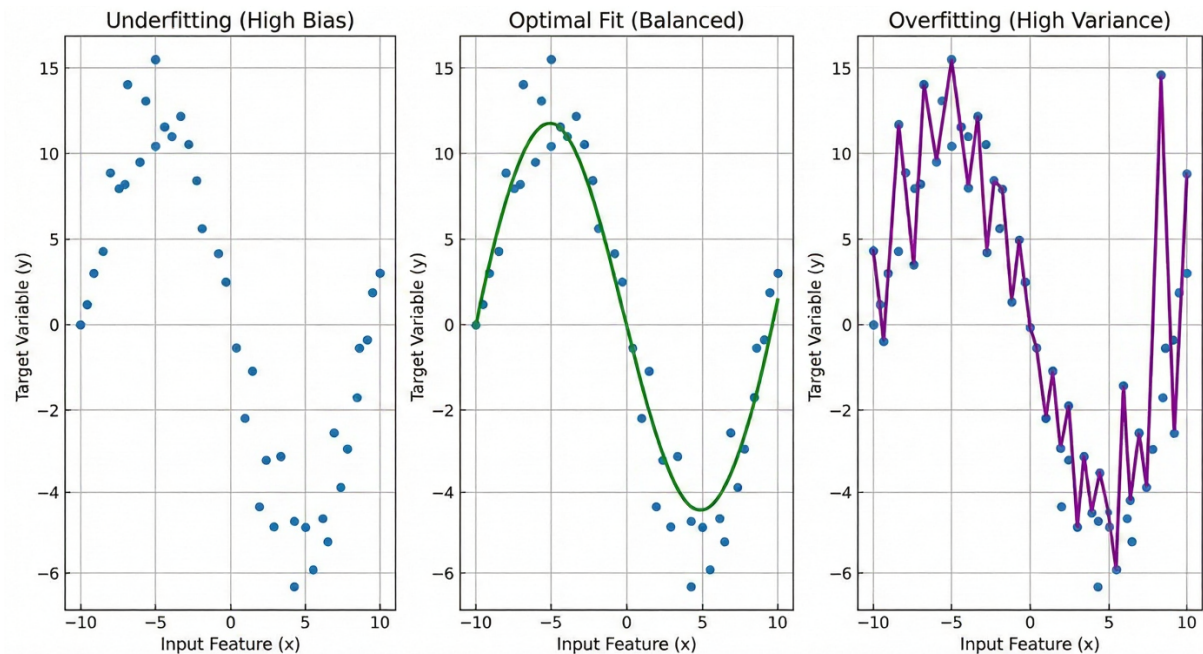
En el entrenamiento de nuestro modelo, el objetivo no es que "recuerde" los datos pasados, sino que **generalice** correctamente ante datos futuros que nunca ha visto. Durante este proceso, nos enfrentamos a dos demonios principales: el sesgo y la varianza.

1.17.1. El Equilibrio: Bias vs Variance

- **Underfitting (Subajuste) - "El Modelo Perezoso":** Ocurre cuando el modelo es demasiado simple para captar la complejidad de los datos. Tiene un alto **Sesgo (Bias)**.
 - *Síntoma:* Tiene mucho error tanto en Entrenamiento como en Test.
 - *Analogía:* Un estudiante que solo se lee la contraportada del libro antes del examen. No aprueba ni los ejercicios de clase.
- **Overfitting (Sobreajuste) - "El Modelo Memori3n":** Ocurre cuando el modelo es demasiado complejo y empieza a memorizar el ruido aleatorio de los datos de entrenamiento en lugar del patr3n real. Tiene alta **Varianza**.
 - *Síntoma:* Error bajísimo en Entrenamiento (99 % acierto), pero error altísimo en Test (50 % acierto).

Analogía: El estudiante que solo memoriza

Un estudiante que memoriza las respuestas exactas (A, B, C...) de los exámenes de años anteriores sin entender la materia. Si le cambias el orden de las preguntas, suspende.



1.17.2. El Protocolo de División de Datos

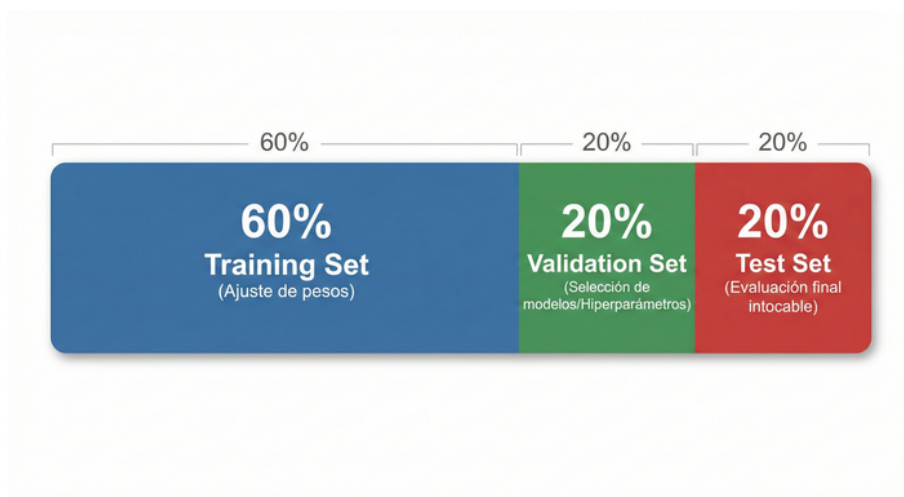
El error más grave de un principiante es ajustar el modelo mirando el examen final (Test). Si usas el Test Set para tomar decisiones, lo has "quemado". Para evitarlo, en entornos profesionales dividimos los datos en **tres** subconjuntos.

Profundidad Técnica: Semilla Aleatoria (Random State)

Antes de dividir, debemos barajar. Para que nuestros experimentos sean reproducibles, fijamos una "semilla" (ej. `random_state=42`). Esto asegura que la baraja de cartas siempre se corte en el mismo punto, permitiéndonos comparar modelos objetivamente.

Las 3 Fases

1. **Training Set (~60-70 %):** *El Libro de Texto.* Se usa para que el algoritmo aprenda los patrones (ajuste sus parámetros internos w).
2. **Validation Set (~15-20 %):** *Los Simulacros de Examen.* Se usa para ajustar los **Hiperparámetros** (el k de KNN, el C de SVM, el Learning Rate). Probamos varias configuraciones aquí y nos quedamos con la que mejor nota saque en Validación. *Importante:* El modelo nunca "entrena" con estos datos, solo lo evaluamos para tomar decisiones de diseño.
3. **Test Set (~15-20 %):** *El Examen Oficial.* Se usa **UNA SOLA VEZ** al final del proyecto. Sirve para dar la métrica final de rendimiento al cliente. Si el resultado es malo aquí, no vale volver atrás; significa que tu diseño falló y debes recomenzar con otra estrategia.



Analogía: El Opositor

- **Train:** Estudias el temario en casa durante meses.
- **Validation:** Haces simulacros semanales. Si sacas mala nota, cambias tu método de estudio (ajustas hiperparámetros).
- **Test:** El día de la oposición. Ya no puedes cambiar tu método de estudio. La nota que saques es la definitiva.

1.17.3. Otras técnicas de validación (Cross-Validation)

¿Qué pasa si tenemos pocos datos (ej. dataset Wine con 178 filas)? Si apartamos un 20 % para validación y otro 20 % para test, nos quedamos sin datos suficientes para que el modelo aprenda. Para solucionar esto, usamos técnicas de **Validación Cruzada**.

K-Fold Cross-Validation (El Estándar)

En lugar de tener un set de validación fijo, rotamos.

1. Dividimos el Train Set en K partes (ej. $K = 5$).
2. Entrenamos 5 veces. En cada vuelta, una parte distinta hace de "Validación" y las otras 4 de "Train".
3. Promediamos las 5 notas resultantes.

Consejo: Si tus clases están desbalanceadas, usa **Stratified K-Fold**, que asegura que en cada pliegue se mantenga la proporción real de clases.

Leave-One-Out Cross-Validation (LOOCV)

Es el caso extremo de K-Fold donde $K = N$ (Número total de datos).

- **Funcionamiento:** Cogemos 1 solo dato para validar y entrenamos con todos los demás. Repetimos esto para CADA dato del dataset.
- **Ventaja:** No tiene sesgo, usamos casi todos los datos para entrenar. Ideal para datasets muy pequeños (< 50 datos).

- **Desventaja:** Coste computacional brutal. Si tienes 1,000 datos, tienes que entrenar el modelo 1,000 veces.

Profundidad Técnica: ¿Cuál elijo?

- **Muchos datos (>10k):** Train/Val/Test simple (Hold-out).
- **Datos medios (<10k):** K-Fold (generalmente $K = 5$ o $K = 10$).
- **Datos minúsculos (<100):** Leave-One-Out.

1.18. Métricas de evaluación

Cuando preguntamos "¿Qué tan bueno es el modelo?", la respuesta novata es mirar la **Accuracy (Exactitud)**.

$$Accuracy = \frac{\text{Aciertos Totales}}{\text{Total de Datos}}$$

La Trampa de la Accuracy

Imagina un dataset de detección de terroristas. 99,99 % son ciudadanos honestos, 0,01 % son terroristas. Un modelo que diga "**Nadie es terrorista**" tendrá una Accuracy del 99,99 %. ¡Parece perfecto! Pero es un modelo inútil porque deja escapar a todos los criminales.

Necesitamos métricas más granulares. Para ello, usamos la **Matriz de Confusión**.

1.18.1. Desglose de la Matriz (Binaria)

- **True Positive (TP):** Predije Cáncer y TIENE Cáncer. (Acierto).
- **True Negative (TN):** Predije Sano y ESTÁ Sano. (Acierto).
- **False Positive (FP - Tipo I):** Predije Cáncer pero está sano. ("Falsa Alarma").
- **False Negative (FN - Tipo II):** Predije Sano pero tiene cáncer. ("Peligro Mortal").

1.18.2. Precision vs Recall (El Dilema)

Rara vez puedes tener ambos al 100 %. Debes elegir qué error toleras más.

Precision (Precisión Pura)

$$Precision = \frac{TP}{TP + FP}$$

Interpretación: "De todos los que yo dije que eran positivos, ¿cuántos lo eran de verdad?"

- **Útil para:** Filtros de Spam.
- **Por qué:** Odiamos los Falsos Positivos. Si el filtro marca un correo importante de tu jefe como Spam (FP), es un desastre. Preferimos dejar pasar un poco de Spam (FN) a cambio de estar seguros de que lo que borramos es basura.

Recall (Sensibilidad / Exhaustividad)

$$Recall = \frac{TP}{TP + FN}$$

Interpretación: "De todos los positivos que existen en la realidad, ¿cuántos fui capaz de encontrar?"

- **Útil para:** Diagnóstico Médico (Cáncer).
- **Por qué:** Odiamos los Falsos Negativos. Si le decimos a un enfermo que está sano (FN), puede morir. Preferimos asustar a un sano por error (FP) y hacerle más pruebas, que dejar escapar a un enfermo.

F1-Score

Es la media armónica entre Precision y Recall. Se usa cuando queremos un equilibrio y no tenemos preferencia clara.

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

1.19. Introducción: Pensamiento Probabilístico

En Regresión intentábamos dibujar una línea. En **Clasificación**, intentamos dibujar fronteras entre grupos. Naive Bayes es una familia de algoritmos "Generativos": intentan modelar cómo se generan los datos de cada clase para luego preguntar: "¿A qué clase pertenece más probablemente este nuevo dato?".

1.20. Desglosando el Teorema de Bayes

El teorema describe la probabilidad de un evento, basado en el conocimiento previo de las condiciones que podrían estar relacionadas con el evento.

$$P(y|X) = \frac{P(X|y) \cdot P(y)}{P(X)}$$

Vamos a diseccionar cada término en el contexto de Machine Learning:

1. **Posterior** $P(y|X)$: *Lo que queremos calcular.* La probabilidad de que la etiqueta sea y (ej. "Spam") dado que hemos visto las características X (ej. palabras del email).
2. **Prior** $P(y)$: *El conocimiento a priori.* ¿Qué tan común es el Spam en general? Si el 90 % de los correos del mundo son Spam, nuestro modelo debería empezar sospechando que todo es Spam antes de leer nada.
3. **Likelihood** $P(X|y)$: *La Verosimilitud.* Si asumimos que el mensaje ES Spam, ¿cuál es la probabilidad de encontrar estas palabras específicas (X) en él? Esto se aprende contando frecuencias en el entrenamiento.
4. **Evidence** $P(X)$: *La constante de normalización.* La probabilidad de que aparezcan estas palabras por pura casualidad. Como es igual para todas las clases (es el denominador común), **podemos ignorarla** para comparar y ahorrar cómputo.

Profundidad Técnica: La Regla de Decisión

En la práctica, calculamos el numerador para la Clase A y para la Clase B, y nos quedamos con la mayor:

$$\hat{y} = \arg \max_y (P(X|y) \cdot P(y))$$

1.21. La Asunción "Naive": Independencia Condicional

El problema real es calcular el Likelihood $P(X|y)$ cuando X tiene muchas características $(x_1, x_2, \dots, x_n)$. Para un email con 2,000 palabras posibles, tendríamos que medir la probabilidad de todas las combinaciones posibles de palabras. Computacionalmente imposible.

El Truco Ingenuo: Asumimos que las características son **independientes** entre sí dado la clase.

$$P(x_1, x_2, \dots, x_n|y) \approx P(x_1|y) \cdot P(x_2|y) \cdot \dots \cdot P(x_n|y)$$

Esto convierte un problema de combinatoria exponencial en una simple multiplicación de probabilidades individuales.

1.22. Variantes del Algoritmo (Tipos de Datos)

Dependiendo de qué tipo de datos tengas en X , usarás una variante distinta de Naive Bayes. Esto es crítico a la hora de escribir código.

1.22.1. Multinomial Naive Bayes (Texto)

Es el clásico para NLP (Procesamiento de Lenguaje Natural).

- **Datos:** Conteos discretos. ¿Cuántas veces aparece la palabra "Dinero" ? (0, 1, 5 veces...).
- **Funcionamiento:** Calcula la probabilidad basada en la frecuencia relativa de los términos.

1.22.2. Bernoulli Naive Bayes (Binario)

- **Datos:** Variables binarias (0 o 1). ¿Aparece la palabra "Dinero"? (Sí/No). No importa si aparece 50 veces, solo si está presente.
- **Uso:** Textos cortos o características de "bandera" (tiene hipoteca / no tiene hipoteca).

1.22.3. Gaussian Naive Bayes (Numérico Continuo)

¡Importante! Este es el que usaremos si el dataset es "Iris" o "Wine" (medidas químicas). Como no podemos "contar" valores continuos (ej. 13.5 grados de alcohol, 13.51, 13.52...), asumimos que los datos siguen una **Distribución Normal (Campana de Gauss)**.

$$P(x_i|y) = \frac{1}{\sqrt{2\pi\sigma_y^2}} \exp\left(-\frac{(x_i - \mu_y)^2}{2\sigma_y^2}\right)$$

El modelo simplemente calcula la Media (μ) y la Desviación Típica (σ) de cada clase y usa la fórmula de la campana para ver "qué tan probable es este número".

1.23. Retos de Ingeniería: El Truco del Logaritmo

Naive Bayes implica multiplicar muchas probabilidades.

$$0,5 \times 0,01 \times 0,002 \times 0,04 \dots$$

El resultado se vuelve un número infinitamente pequeño. Los ordenadores tienen un límite de precisión (*floating point underflow*); si el número es muy pequeño, el ordenador lo redondea a 0 y el modelo deja de funcionar.

La Solución: Sumar Logaritmos Recordamos la propiedad matemática: $\log(a \cdot b) = \log(a) + \log(b)$. En lugar de multiplicar probabilidades, sumamos sus logaritmos.

$$\log(P(y|X)) \propto \log(P(y)) + \sum \log(P(x_i|y))$$

Esto mantiene los números en una escala manejable (ej. -50.4 en lugar de $1e^{-23}$) y la clasificación sigue siendo correcta porque el logaritmo es una función monótona (mantiene el orden).

1.24. El Problema de la Probabilidad Cero (Laplace Smoothing)

Si en fase de test aparece una palabra que nunca vimos en entrenamiento (ej. "ChatGPT"), su probabilidad condicional es 0. Al multiplicar, anula toda la predicción.

$$P(\text{Spam}) = 0,5 \times 0,01 \times \mathbf{0} \times 0,8 = 0$$

Suavizado de Laplace: Fingimos que todas las palabras del diccionario han aparecido **al menos una vez** extra.

$$P(w|c) = \frac{\text{count}(w, c) + \alpha}{\text{count}(c) + \alpha \cdot \text{Vocabulario}}$$

Generalmente $\alpha = 1$. Esto asegura que ninguna probabilidad sea jamás 0 absoluto, dando al modelo robustez ante datos nuevos.

1.25. Introducción: El Arquitecto Geométrico

Mientras que Naive Bayes se basaba en la probabilidad estadística, las **Máquinas de Vectores de Soporte (SVM)** se basan en la geometría espacial. Es considerado uno de los algoritmos más elegantes y robustos de la historia del Machine Learning (reinó en los años 90 antes del auge del Deep Learning).

El objetivo: Encontrar el **Hiperplano Óptimo** que separe las clases.

- En 2D (dos variables), el hiperplano es una **Línea**.
- En 3D (tres variables), es un **Plano** plano (como una hoja de papel).
- En n dimensiones, es un **Hiperplano**.

1.26. El Concepto de Margen Máximo

Imagina que tienes puntos rojos y azules separados en un papel. Puedes dibujar infinitas líneas rectas que los separen. ¿Cuál eliges?

SVM no busca cualquier línea. Busca la línea que pasa exactamente por el medio de la "carretera" más ancha posible que se pueda dibujar entre las dos clases.

1.26.1. Componentes Clave:

1. **Hiperplano de Decisión:** La línea central ($w \cdot x + b = 0$).
2. **Márgenes:** Las líneas paralelas a los lados de la carretera. La distancia entre ellas es lo que queremos **maximizar**.
3. **Vectores de Soporte (Support Vectors):** Aquí está la clave del nombre. Son los puntos de datos (instancias) que "tocan" los márgenes. Son los puntos más difíciles de clasificar (los que están en la frontera).

Profundidad Técnica: La dispersión no importa (Sparsity)

A diferencia de la Regresión Lineal, que intenta ajustarse a la media de todos los puntos, a SVM **no le importan los puntos que están lejos de la frontera**. Si mueves un punto rojo que está muy lejos del margen, la línea no se mueve. El modelo se sostiene *únicamente* sobre los Vectores de Soporte. Esto lo hace muy eficiente en memoria una vez entrenado.

1.27. Realidad vs Teoría: Hard Margin vs Soft Margin

En un mundo ideal, los datos son linealmente separables (puedes separarlos con una línea perfecta). Pero el mundo real es sucio. Siempre hay un punto azul metido en la zona de los rojos (ruido).

1.27.1. El Hiperparámetro C (Regularización)

Para gestionar esto, SVM tiene un parámetro crucial llamado C que controla la rigidez del margen.

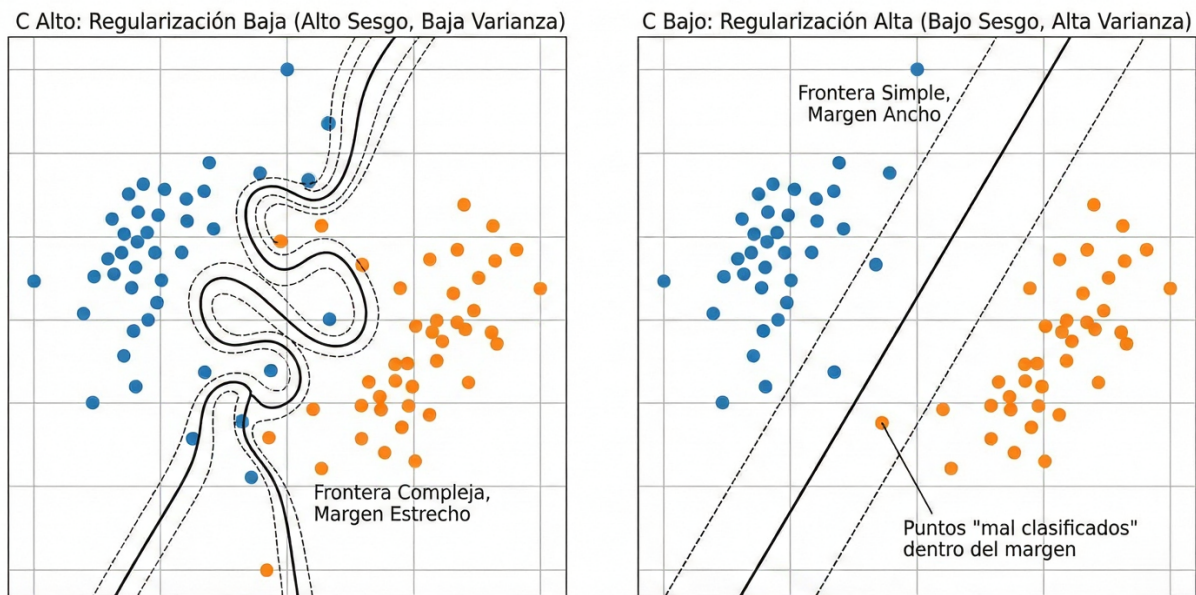
- **C Alto (Hard Margin / Estricto):** "No tolero errores". El modelo intentará clasificar correctamente el 100 % de los puntos de entrenamiento, aunque tenga que hacer un margen muy estrecho y zigzagueante.
 - *Riesgo:* Alto **Overfitting**. El modelo no generalizará bien.
- **C Bajo (Soft Margin / Permisivo):** "Prefiero una carretera ancha aunque me equivoque en algunos puntos". Permite que algunos puntos invadan el margen para mantener la frontera simple.
 - *Ventaja:* Mejor generalización. Más robusto al ruido.

1.28. The Kernel Trick: La Magia Dimensional

¿Qué pasa si los datos NO se pueden separar con una línea recta? (Ejemplo: Un punto rojo rodeado de un anillo de puntos azules). En 2D es imposible separarlos linealmente.

Aquí entra la genialidad de SVM: **Proyectar los datos a una dimensión superior**.

Efecto del Hiperparámetro C en SVM



Analogía: La Hoja de Papel

Imagina que dibujas un punto rojo en el centro de una hoja y un círculo de puntos azules alrededor. No puedes separarlos con una regla (línea recta) sin romper el papel. **Solución:** Golpea la mesa. La hoja vibra y el punto rojo (en el centro) salta hacia arriba (eje Z). En ese instante, mientras el punto rojo vuela por encima de los azules, puedes pasar una lámina de metal (un plano) por debajo de él. Al volver a mirar la hoja desde arriba (proyectar a 2D), ese plano se ve como un círculo perfecto.

1.28.1. Tipos de Kernels

En `scikit-learn`, cambiamos esto con el parámetro `kernel=...`

1. **Linear:** No hace magia. Útil para textos (muchas dimensiones) o datos simples.
2. **Polynomial:** Crea curvas polinómicas.
3. **RBF (Radial Basis Function):** *El estándar por defecto.* Crea "montañas" alrededor de los puntos. Tiene una capacidad de adaptación infinita.

1.29. El Hiperparámetro Gamma (γ)

Si usas el Kernel RBF, aparece un segundo parámetro vital: **Gamma**. Controla el "alcance" de la influencia de un solo punto de entrenamiento.

- **Gamma Alto:** Cada punto tiene un aura muy pequeña. El modelo crea "islotos" o manchas muy específicas alrededor de los puntos.
 - *Resultado:* Fronteras muy complejas y fragmentadas. **Overfitting.**
- **Gamma Bajo:** Cada punto tiene una influencia enorme. Las auras se mezclan creando fronteras muy suaves.
 - *Resultado:* Fronteras casi lineales. **Underfitting.**

Profundidad Técnica: Matemáticamente

En el kernel RBF ($K(x, x') = \exp(-\gamma \|x - x'\|^2)$), gamma es el inverso de la desviación estándar de la campana de Gauss.

γ alto $\rightarrow$ Campana estrecha (Pico de aguja).
 γ bajo $\rightarrow$ Campana ancha (Colina suave).

1.30. Resumen de Ajuste (Tuning Guide)

Esta tabla salvará tu modelo durante la práctica:

Parámetro	Valor Bajo	Valor Alto
C (Rigidez)	Margen suave, permite errores. (<i>Tiende a Underfitting</i>)	Margen duro, estricto. (<i>Tiende a Overfitting</i>)
Gamma (Alcance) (Solo RBF)	Influencia lejana, borde suave. (<i>Tiende a Underfitting</i>)	Influencia cercana, borde irregular. (<i>Tiende a Overfitting</i>)

Capítulo 2

Sesión 2 : Árboles de decisión, Ensembles y Clustering

2.1. Árboles de Decisión (Decision Trees)

A diferencia de los modelos lineales (SVM, Regresión Logística) que buscan separar el espacio mediante un hiperplano geométrico global, los árboles de decisión utilizan una estrategia **no paramétrica** y **jerárquica**. Su enfoque consiste en dividir el espacio de características en rectángulos (o hiper-rectángulos) mediante reglas lógicas secuenciales (*if – else*).

2.1.1. Fundamentos y Anatomía

Un árbol de decisión es, formalmente, un grafo acíclico dirigido que modela decisiones y sus posibles consecuencias. Se compone de tres tipos de nodos:

- **Nodo Raíz (Root Node):** Representa la población total del dataset. Es el punto de partida donde el algoritmo busca la primera división óptima.
- **Nodos Internos (Decision Nodes):** Puntos intermedios donde el algoritmo evalúa una característica específica (X_j) con un umbral de corte (t_k).

Ejemplo: ¿Es la variable "Ingresos" > 30,000?

- **Hojas (Leaf Nodes):** Nodos terminales que ya no se dividen más. Contienen la decisión final:
 - *En Clasificación:* La clase mayoritaria en ese nodo (Moda).
 - *En Regresión:* El valor promedio de las muestras que caen en ese nodo (Media).

2.1.2. El Algoritmo CART (Classification and Regression Trees)

La implementación estándar en bibliotecas como `scikit-learn` se basa en el algoritmo **CART**. Este algoritmo sigue un enfoque **Greedy** (codicioso): en cada paso, busca la división que maximice la ganancia de información *localmente*, sin preocuparse por si esa decisión será la mejor a largo plazo (no garantiza el óptimo global, pero es computacionalmente eficiente).

La Matemática de la División (Splitting)

El algoritmo busca iterativamente un par (k, t_k) —donde k es el índice de una característica y t_k es un umbral— que divida el conjunto de datos D en dos subconjuntos D_{left} y D_{right} .

La función de coste $J(k, t_k)$ que el algoritmo intenta **minimizar** es la impureza ponderada de los hijos:

$$J(k, t_k) = \frac{m_{left}}{m} G_{left} + \frac{m_{right}}{m} G_{right} \quad (2.1)$$

Donde:

- $G_{left/right}$: Impureza del subconjunto izquierdo/derecho.
- $m_{left/right}$: Número de instancias en el subconjunto izquierdo/derecho.
- m : Número total de instancias en el nodo padre.

2.1.3. La familia Quinlan (ID3 y C4.5)

Aunque `scikit-learn` utiliza CART, históricamente existen otros algoritmos fundamentales desarrollados por Ross Quinlan que introdujeron conceptos clave.

- **ID3 (Iterative Dichotomiser 3)**: El precursor. Solo funcionaba con variables categóricas y usaba la Entropía. Tenía un defecto grave: tendía a seleccionar variables con muchos niveles (ej. DNI o ID), que tienen alta pureza pero nula capacidad de generalización.
- **C4.5**: La evolución de ID3 y rival directo de CART. Sus diferencias clave con la implementación moderna de Python son:
 1. **Divisiones Múltiples (Multi-way Splits)**: A diferencia de CART (que siempre es binario), C4.5 puede dividir un nodo en N hijos, uno por cada categoría de la variable.
 2. **Ratio de Ganancia (Gain Ratio)**: Para corregir el sesgo de ID3, normaliza la Ganancia de Información dividiéndola por la entropía intrínseca de la variable.
 3. **Manejo de Nulos**: C4.5 puede manejar valores faltantes distribuyendo probabilísticamente la instancia por las ramas, mientras que CART estándar en `sklearn` requiere imputación previa.

CART vs C4.5 en Producción

En la actualidad, **CART** ha ganado la batalla por eficiencia computacional. Las divisiones binarias son más rápidas de procesar en hardware y menos propensas a fragmentar los datos excesivamente rápido (lo que causaría overfitting).

2.1.4. Criterios de Impureza (Clasificación)

Para resolver problemas de clasificación, necesitamos una métrica que cuantifique el "desorden" o la mezcla de clases en un nodo. Un nodo es **puro** si todas sus muestras pertenecen a una única clase.

A. Impureza de Gini (Gini Impurity)

Es el criterio por defecto en CART debido a su eficiencia computacional (evita el cálculo de logaritmos). Mide la probabilidad de clasificar incorrectamente un elemento aleatorio si se etiquetara aleatoriamente según la distribución de clases del nodo.

$$Gini = 1 - \sum_{i=1}^C (p_i)^2 \quad (2.2)$$

Donde p_i es la proporción de muestras de la clase i en el nodo.

- **Mínimo (0.0)**: Nodo totalmente puro.
- **Máximo (0.5)**: Máxima impureza (en clasificación binaria, 50 % clase A y 50 % clase B).

B. Entropía (Information Gain)

Basada en la teoría de la información de Shannon. Mide el nivel de incertidumbre o sorpresa del sistema.

$$H = - \sum_{i=1}^C p_i \log_2(p_i) \quad (2.3)$$

Aunque teóricamente más robusta, en la práctica genera árboles muy similares a Gini. La principal diferencia es que la Entropía tiende a generar árboles ligeramente más balanceados, pero es más lenta de calcular.

2.1.5. Árboles de Regresión

Los árboles también pueden predecir valores continuos. En este caso, no usamos Gini ni Entropía, sino la **Reducción de Varianza**.

El criterio de división busca minimizar el **Error Cuadrático Medio (MSE)** dentro de los nodos hijos:

$$MSE_{nodo} = \frac{1}{m_{nodo}} \sum_{i=1}^{m_{nodo}} (y_i - \hat{y}_{nodo})^2 \quad (2.4)$$

Donde $\hat{y}_{nodo}$ es la media de los valores objetivo (y) de las muestras en ese nodo. El árbol intentará agrupar valores numéricos similares para reducir la dispersión (varianza) dentro de cada hoja.

2.1.6. Regularización: El Problema del Overfitting

Los árboles de decisión son algoritmos de **Bajo Bias y Alta Varianza**. Son modelos "ca-maleónicos": pueden adaptar su estructura a cualquier complejidad de los datos, incluido el ruido.

Si no se limitan, crecerán indefinidamente hasta que cada hoja contenga una sola muestra, logrando un error de entrenamiento de 0 pero fallando estrepitosamente en generalización (*Overfitting*).

Para evitar esto, aplicamos **Poda (Pruning)** mediante hiperparámetros clave de ingeniería:

- **max_depth**: La restricción más potente. Limita la profundidad máxima del árbol.
- **min_samples_split**: Número mínimo de muestras necesarias para permitir que un nodo interno se divida. (Ej: Si es 20, un nodo con 15 muestras se convierte forzosamente en hoja).
- **min_samples_leaf**: Número mínimo de muestras que debe tener una hoja. Esto "suaviza" el modelo, impidiendo que aísle outliers individuales.
- **max_features**: Número máximo de características a considerar para buscar la mejor división en cada nodo (Crucial para Random Forest).

2.1.7. Interpretabilidad: Feature Importance

Una ventaja crítica de los árboles en la industria es que son modelos de "Caja Blanca". Podemos cuantificar matemáticamente qué características son las más relevantes para la predicción.

La **Importancia de Gini** (Mean Decrease in Impurity) se calcula sumando cuánto reduce la impureza una característica específica (X_j) en todos los nodos donde se utiliza, ponderado por el número de muestras que pasan por esos nodos.

$$Importancia(X_j) = \frac{\sum_{n \in \text{Nodos}(X_j)} N_n \Delta Gini_n}{\sum_{k \in \text{Todas}} Importancia(X_k)} \quad (2.5)$$

Esto permite al ingeniero realizar **Selección de Características** (Feature Selection) basada en el propio modelo, descartando variables que el árbol nunca (o casi nunca) elige para dividir los datos.

2.2. Métodos de Ensamblaje (Ensemble Learning)

En la ingeniería de ML moderna, rara vez confiamos la decisión crítica a un único modelo. La intuición detrás del ensamblaje es la **Sabiduría de las Masas** (Wisdom of the Crowd): un conjunto de aprendices débiles (*Weak Learners*) puede combinarse para formar un aprendiz fuerte (*Strong Learner*) con mejor capacidad de generalización.

Matemáticamente, los ensamblajes buscan alterar la ecuación del error total:

$$Error = Bias^2 + Varianza + Ruido \quad (2.6)$$

- **Bagging:** Se enfoca en reducir la **Varianza** (Overfitting).
- **Boosting:** Se enfoca en reducir el **Bias** (Underfitting).

2.2.1. Bagging (Bootstrap Aggregating)

El concepto central es el entrenamiento en **paralelo** de múltiples modelos independientes, donde cada uno ve una versión ligeramente distorsionada de la realidad. La predicción final es el promedio (regresión) o el voto por mayoría (clasificación).

El Mecanismo de Bootstrapping

Dado un dataset D de tamaño m , generamos B nuevos datasets $D_1, D_2, \dots, D_B$ mediante muestreo aleatorio **con reemplazo**. Esto implica que, en un subconjunto D_i , algunas instancias originales pueden aparecer varias veces, mientras que otras pueden no aparecer en absoluto.

Random Forest: La Evolución del Bagging

Un *Random Forest* es una mejora sobre el Bagging de árboles de decisión estándar. Introduce una segunda capa de aleatoriedad para **descorrelacionar** los árboles.

1. **Muestreo de Filas (Bootstrap):** Cada árbol se entrena con un subconjunto de datos diferente.
2. **Muestreo de Columnas (Feature Randomness):** En cada nodo del árbol, en lugar de buscar la mejor división entre *todas* las características (p), el algoritmo selecciona un subconjunto aleatorio de características ($k < p$) y busca la mejor división solo entre ellas.

¿Por qué funciona? Si hay una característica muy fuerte (ej. "Ingresos" para predecir "Crédito"), todos los árboles estándar la elegirían en la raíz, volviéndose idénticos (alta correlación). Al forzar a algunos árboles a ignorar esa característica, el bosque explora patrones más sutiles y reduce la varianza global.

Evaluación Out-of-Bag (OOB)

En ingeniería, el Bagging nos regala una métrica de validación gratuita sin necesidad de un set de validación separado. La probabilidad de que una instancia específica *no* sea seleccionada en una muestra bootstrap de tamaño m es:

$$P(\text{no_seleccion}) = \left(1 - \frac{1}{m}\right)^m \approx \frac{1}{e} \approx 0,368 \quad (2.7)$$

Esto significa que aproximadamente el **36.8 %** de los datos no son vistos por cada árbol. Podemos usar estos datos "sobrantes" para evaluar el rendimiento del árbol, promediando el error sobre todo el bosque.

2.2.2. Boosting (Hypothesis Boosting)

A diferencia del Bagging (paralelo), el Boosting es un proceso **secuencial**. La idea es construir una cadena de modelos donde cada uno se especializa en corregir los errores de su predecesor. Convierte aprendices débiles (ligeramente mejores que el azar) en uno fuerte.

AdaBoost (Adaptive Boosting)

El primer algoritmo de boosting exitoso. Funciona asignando **pesos** (w_i) a cada instancia.

1. Entrena un modelo base h_1 .
2. Aumenta el peso w_i de las instancias mal clasificadas por h_1 .
3. Entrena h_2 obligándolo a prestar más atención a esas instancias difíciles.
4. La predicción final es una suma ponderada de todos los modelos.

Gradient Boosting (La Joya de la Corona)

Es una generalización matemática más robusta. En lugar de reponderar instancias, Gradient Boosting entrena nuevos modelos para predecir los **Residuos** (el error) del modelo anterior.

Sea $F_{m-1}(x)$ la predicción del ensamblaje en la etapa $m - 1$. El objetivo del nuevo modelo $h_m(x)$ es minimizar la función de pérdida L :

$$h_m(x) \approx -\frac{\partial L(y, F_{m-1}(x))}{\partial F_{m-1}(x)} \quad (2.8)$$

El término de la derecha es el **Gradiente Negativo** (la dirección de mayor descenso del error).

Intuición Ingenieril: Si el modelo 1 predice 80 y la realidad es 100, el residuo es +20. El modelo 2 no intentará predecir 100, intentará predecir +20. La suma final será $80 + 20 = 100$.

2.2.3. Implementaciones Industriales (SOTA)

En competencias y producción (datos tabulares), raramente usamos la implementación básica de **sklearn**. Usamos librerías optimizadas que aplican regularización avanzada y optimización de hardware:

▪ XGBoost (Extreme Gradient Boosting):

- Utiliza derivadas de segundo orden (Hessiano) para una aproximación más precisa de la función de coste (Serie de Taylor).

- Incluye regularización L1/L2 dentro de la función objetivo del árbol para evitar overfitting.
- Maneja valores nulos (Sparsity-aware) aprendiendo una dirección por defecto.

■ **LightGBM (Microsoft):**

- Utiliza *Gradient-based One-Side Sampling (GOSS)* para ignorar instancias con gradientes pequeños (ya aprendidas), acelerando el entrenamiento.
- Crece los árboles por hojas (*Leaf-wise*) en lugar de por niveles, lo que reduce más el error pero puede sobreajustar si hay pocos datos.

■ **CatBoost (Yandex):**

- Especializado en **características categóricas**. Aplica *Ordered Target Statistics* para codificar categorías sin causar Data Leakage.
- Utiliza árboles simétricos (oblivious trees) que son muy rápidos en inferencia.

2.2.4. Stacking (Stacked Generalization)

Es una técnica de ensamblaje heterogéneo.

1. **Nivel 0 (Base Learners):** Entrenamos modelos distintos (ej. un SVM, un Random Forest y una Red Neuronal) sobre el mismo dataset.
2. **Nivel 1 (Meta-Learner):** Las predicciones de los modelos del Nivel 0 se convierten en las *nuevas características de entrada* para un modelo final (generalmente una Regresión Lineal o Logística).

Alerta de Ingeniería: El Stacking es extremadamente propenso al Data Leakage. Es obligatorio usar *Cross-Validation* para generar las predicciones del Nivel 0.

2.2.5. Nota de Ingeniería: Estrategias Multiclase (OvR vs OvO)

Muchos algoritmos de la “caja de herramientas” (SVM, Regresión Logística) son nativamente **binarios**: solo saben separar “Clase A” de “Clase B”. ¿Cómo los usamos para clasificar 10 dígitos o 3 tipos de vino? Utilizamos meta-estrategias que descomponen el problema multiclase en múltiples problemas binarios.

One-vs-Rest (OvR) / One-vs-All

Entrenamos un modelo por cada clase.

- **Estrategia:** Para N clases, entrenamos N clasificadores.
- **El Duelo:** El Clasificador 1 aprende a distinguir “Clase 1” vs “Todo lo demás”.
- **Predicción:** Ejecutamos los N modelos. La clase ganadora es la que obtiene la probabilidad (o puntuación de confianza) más alta.
- **Ventaja:** Computacionalmente eficiente (solo N modelos).
- **Desventaja:** *Desbalanceo inducido*. Si tenemos 3 clases equilibradas (33% cada una), al entrenar “Clase 1 vs Resto”, el modelo ve un ratio de 1:2.

One-vs-One (OvO)

Entrenamos un modelo por cada *par* de clases.

- **Estrategia:** Entrenamos un clasificador para distinguir Clase 1 vs Clase 2", otro para Clase 1 vs Clase 3", etc.
- **Número de Modelos:** Crece cuadráticamente: $\frac{N(N-1)}{2}$.

Ejemplo: Para 10 dígitos (MNIST), OvR entrena 10 modelos. OvO entrena 45 modelos.

- **Predicción:** Sistema de votación. Cada modelo emite un voto por su clase ganadora. La clase con más votos gana.
- **Ventaja:** Cada sub-problema es equilibrado y más pequeño. Es vital para algoritmos que escalan mal con el número de datos ($O(n^3)$), como SVM con Kernel.

Decisión de Diseño

scikit-learn elige automáticamente por ti, pero debes saber qué está haciendo:

- **SVM (SVC):** Usa **One-vs-One** porque el algoritmo es lento con muchos datos. Prefiere muchos modelos pequeños a pocos grandes.
- **Regresión Logística / Lineal:** Usa **One-vs-Rest** por eficiencia.

2.3. Aprendizaje No Supervisado (Clustering)

En el aprendizaje supervisado, el objetivo era encontrar un mapeo $f : X \rightarrow y$ que minimizara un error de predicción. En el aprendizaje no supervisado, abandonamos la variable objetivo y . Solo disponemos de X .

El objetivo de la ingeniería aquí cambia radicalmente: buscamos describir la **estructura subyacente** o la densidad de probabilidad $P(X)$ que generó los datos. Las aplicaciones principales son la segmentación (marketing), la detección de anomalías (seguridad) y la reducción de dimensionalidad.

2.3.1. K-Means Clustering

Es el algoritmo de agrupamiento más utilizado por su eficiencia ($O(n)$) y simplicidad, aunque se basa en fuertes suposiciones geométricas.

Formulación Matemática (Inercia)

K-Means es un problema de optimización. Busca dividir las n observaciones en K clústers ($C_1, \dots, C_K$) para minimizar la **Suma de Cuadrados Intra-Clúster** (Within-Cluster Sum of Squares, WCSS), también llamada **Inercia**:

$$\min_{C_1, \dots, C_K} \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2 \quad (2.9)$$

Donde μ_k es el centroide (media) del clúster C_k .

El Algoritmo EM (Expectation-Maximization)

Como encontrar la solución óptima global es un problema NP-Hard, K-Means utiliza una aproximación iterativa que converge a un óptimo local:

1. **Inicialización:** Seleccionar K centroides aleatorios ($\mu^{(0)}$).
2. **Paso E (Asignación):** Asignar cada punto x_i al centroide más cercano (distancia Euclidiana mínima).
3. **Paso M (Actualización):** Recalcular los centroides $\mu^{(t+1)}$ como el promedio de los puntos asignados a cada clúster.
4. **Convergencia:** Repetir hasta que los centroides no cambien ($\Delta\mu < \epsilon$).

Limitaciones de Ingeniería

- **Suposición de Esfericidad:** K-Means asume que los clústers son convexos e isotrópicos (burbujas redondas). Falla estrepitosamente en formas irregulares (ej. lunas entrelazadas).
- **Sensibilidad a la Escala:** Al usar distancia Euclidiana, es **obligatorio** estandarizar los datos (Z-Score) antes de ejecutarlo. De lo contrario, las variables con mayor magnitud dominarán la geometría.
- **Inicialización:** Una mala elección inicial de centroides puede llevar a resultados subóptimos. La solución estándar es **K-Means++**, que inicializa los centroides dispersándolos lo máximo posible.

2.3.2. Clustering Jerárquico (Aglomerativo)

A diferencia de K-Means, este enfoque no requiere predefinir K . Construye una jerarquía de clústers que podemos visualizar como un árbol o **Dendrograma**.

Estrategia Bottom-Up

1. Inicialmente, cada punto es su propio clúster (N clústers).
2. En cada paso, fusionamos los dos clústers más cercanos.
3. Repetimos hasta que solo queda un clúster gigante.

Criterios de Enlace (Linkage)

La clave matemática es cómo definimos la "distancia" entre dos grupos de puntos:

- **Ward:** Minimiza la varianza total dentro de los clústers fusionados. Tiende a crear grupos de tamaño similar (similar a K-Means).
- **Single Linkage:** Distancia mínima entre cualquier par de puntos. Puede sufrir el "efecto cadena" (clústers alargados).
- **Complete Linkage:** Distancia máxima entre puntos. Tiende a crear clústers compactos.

2.3.3. DBSCAN (Density-Based Spatial Clustering)

DBSCAN representa un cambio de paradigma: define los clústers como **regiones de alta densidad** separadas por regiones de baja densidad.

Parámetros e Hiperparámetros

Requiere dos parámetros críticos:

- ε (Epsilon): Radio de vecindad alrededor de un punto.
- *MinPts*: Número mínimo de puntos dentro de ε para considerar una zona "densa".

Clasificación de Puntos

El algoritmo clasifica cada instancia en tres categorías:

1. **Núcleo (Core Point)**: Tiene al menos *MinPts* vecinos en su radio ε .
2. **Borde (Border Point)**: No llega a *MinPts*, pero es vecino de un punto Núcleo.
3. **Ruido (Noise)**: Ni es núcleo ni es borde.

Ventajas Críticas en Ingeniería

- **Robustez a Outliers**: A diferencia de K-Means, que fuerza a todo punto a pertenecer a un clúster (distorsionando el centroide), DBSCAN marca los puntos anómalos como "Ruido" (-1) y los ignora.
- **Topología Arbitraria**: Puede encontrar clústers con formas no convexas (anillos, serpientes) que K-Means no puede modelar.

2.3.4. Validación de Clústers

Al no tener etiquetas (y_{true}), no podemos usar Accuracy. Usamos métricas de estructura interna.

Método del Codo (Elbow Method)

Específico para K-Means. Graficamos la Inercia vs K . Buscamos el punto de inflexión donde añadir más clústers deja de reducir significativamente la varianza (rendimientos decrecientes).

Coeeficiente de Silueta (Silhouette Score)

Mide qué tan similar es un punto a su propio clúster (cohesión) comparado con otros clústers (separación).

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \quad (2.10)$$

- $a(i)$: Distancia media intra-clúster.
- $b(i)$: Distancia media al clúster vecino más cercano.
- $s \approx +1$: Clúster denso y bien separado (Ideal).
- $s \approx 0$: Solapamiento de clústers.
- $s \approx -1$: Asignación incorrecta.

2.4. Optimización de Hiperparámetros

Un modelo de ML tiene dos tipos de parámetros:

- **Parámetros Internos:** Los aprende el modelo solo durante el `.fit()` (ej. los pesos w en regresión, los puntos de corte en un árbol).
- **Hiperparámetros:** Los configura el ingeniero *antes* del entrenamiento (ej. K en K-Means, `max_depth` en árboles, λ en regularización).

El objetivo de este módulo es encontrar la combinación de hiperparámetros que maximice la métrica de validación, un proceso conocido como *Hyperparameter Tuning*.

2.4.1. Grid Search (Búsqueda en Rejilla)

Es el método de fuerza bruta. Definimos un "espacio de búsqueda" discreto para cada hiperparámetro y evaluamos el modelo en **todas** las combinaciones posibles (Producto Cartesiano).

Ejemplo: Si probamos 3 profundidades y 4 criterios de división, entrenaremos $3 \times 4 = 12$ modelos.

Ventaja: Garantiza encontrar el mejor modelo dentro de la rejilla definida. **Desventaja:** Su coste computacional es exponencial ($O(n^k)$). Inviabile con muchos hiperparámetros.

2.4.2. Random Search (Búsqueda Aleatoria)

En lugar de probar todas las combinaciones, seleccionamos configuraciones al azar dentro del espacio de búsqueda durante un número fijo de iteraciones (n_{iter}).

Teorema de la Eficiencia: Empíricamente, Random Search es más eficiente que Grid Search. Si un hiperparámetro es irrelevante para el problema, Grid Search desperdicia tiempo probando sus variantes, mientras que Random Search explora más valores de los hiperparámetros que sí importan.

2.4.3. Optimización Bayesiana (Optuna / Hyperopt)

El estado del arte en ingeniería. A diferencia de Grid/Random (que no aprenden de sus errores), la Optimización Bayesiana utiliza los resultados de las iteraciones pasadas para predecir dónde es más probable encontrar un mejor resultado en la siguiente iteración. Construye un modelo probabilístico (Surrogate Model) de la función objetivo y equilibra:

- **Exploración:** Probar zonas desconocidas (alta incertidumbre).
- **Explotación:** Probar cerca de los mejores resultados conocidos.

2.5. Ingeniería de Producción (Pipelines)

El paso de un cuaderno de experimentos (Jupyter) a un sistema en producción es donde fallan el 80 % de los proyectos. El error más común es la inconsistencia en el preprocesamiento: aplicar una limpieza distinta en entrenamiento y en predicción.

2.5.1. El Concepto de Pipeline

Un **Pipeline** en **scikit-learn** es un objeto que encapsula secuencialmente todos los pasos de transformación de datos y el estimador final.

$$\text{Pipeline} = [\text{Transform}_1 \rightarrow \text{Transform}_2 \rightarrow \dots \rightarrow \text{Modelo}] \quad (2.11)$$

Ventajas Críticas:

1. **Prevención de Data Leakage:** Al usar `cross_val_score` sobre un Pipeline, el escalado y la imputación se ajustan automáticamente solo en los *folds* de entrenamiento, protegiendo la validación.
2. **Reproducibilidad:** Garantiza que los datos nuevos en producción sufran exactamente las mismas transformaciones (en el mismo orden) que los datos de entrenamiento.
3. **Código Limpio:** Reemplaza docenas de líneas de código disperso por una sola llamada a `pipeline.fit()` y `pipeline.predict()`.

2.5.2. Column Transformer

En la vida real, los datasets son heterogéneos (columnas numéricas, categóricas, texto). No podemos aplicar **StandardScaler** al texto ni **OneHotEncoder** a la edad. El **ColumnTransformer** permite aplicar ramas de preprocesamiento paralelas:

- **Rama A (Numérica):** Imputación de Media → Escalado.
- **Rama B (Categórica):** Imputación de Moda → One-Hot Encoding.

Luego, une los resultados en una sola matriz para el modelo.

2.5.3. Serialización (Persistencia del Modelo)

Una vez entrenado el modelo (o pipeline), necesitamos guardarlo en disco para usarlo en una API o aplicación web sin re-entrenar.

Pickle y Joblib

Son los formatos estándar en Python para serializar objetos binarios.

- **Joblib:** Optimizado para arrays de NumPy grandes (preferido en ML).
- **Pickle:** Estándar de Python.

Riesgo de Seguridad: Nunca cargues un archivo `.pkl` de una fuente desconocida; puede ejecutar código malicioso al deserializarse.

ONNX (Open Neural Network Exchange)

Para entornos de producción de alto rendimiento (C++, Java, Edge Devices), no usamos Pickle. Exportamos el modelo a formato **ONNX**, un estándar abierto que permite ejecutar modelos de **scikit-learn** o **PyTorch** en cualquier hardware de manera optimizada e independiente del lenguaje original.

2.6. Procesamiento de Datos Desbalanceados

En problemas reales (fraude bancario, diagnóstico de enfermedades raras, predicción de clics), la clase de interés (Positiva) suele ser una minoría extrema (ej. 1 % vs 99 %). Los algoritmos estándar asumen que las clases están equilibradas. Si no lo están, optimizarán la *Accuracy* prediciendo siempre la clase mayoritaria (el "modelo perezoso").

2.6.1. Técnicas de Re-Muestreo (Resampling)

Modificamos el dataset de entrenamiento para engañar al modelo y mostrarle un escenario equilibrado. **Importante:** Solo se aplica al *Train Set*, nunca al Test.

Undersampling (Submuestreo)

Eliminamos aleatoriamente instancias de la clase mayoritaria hasta igualar a la minoritaria.

- **Ventaja:** El entrenamiento es rapidísimo (menos datos).
- **Riesgo:** Perdemos información valiosa de la clase mayoritaria.

Oversampling (Sobremuestreo)

Duplicamos instancias de la clase minoritaria.

- **Riesgo:** Overfitting extremo. El modelo memoriza las instancias duplicadas.

2.6.2. Generación de Datos Sintéticos (SMOTE)

Synthetic Minority Over-sampling Technique. En lugar de duplicar, SMOTE crea "personas falsas" que son matemáticamente plausibles.

1. Toma un punto de la clase minoritaria (A).
2. Busca sus K vecinos más cercanos de la misma clase (B).
3. Traza una línea entre A y B y crea un punto sintético en un lugar aleatorio de esa línea.

2.6.3. Class Weights (Pesos de Clase)

La solución más elegante en ingeniería (sin tocar los datos). Modificamos la Función de Coste (J) del algoritmo para que castigue más los errores en la clase minoritaria.

$$J_{total} = w_0 \cdot Error_{Clase0} + w_1 \cdot Error_{Clase1} \quad (2.12)$$

Si la Clase 1 es el 1 % de los datos, le asignamos un peso $w_1 = 99$. Así, equivocarse en un solo caso positivo duele tanto como equivocarse en 99 negativos. En `scikit-learn` y `XGBoost`, esto se activa con el parámetro `class_weight='balanced'` o `scale_pos_weight`.